

OmegaBuzz

A Human-Agent Collaboration and Cognitive Hive Platform

One integrated place to remember, coordinate, govern, and improve

OMEGABUZZ extends Block's Buzz collaboration substrate with OmegaHive organizational control, layered AtomSpace memory, and Scout-style provenance-first navigation. It is designed as a practical home for individual OmegaClaw agents, complete hives, and networks of cooperating hives.

Document status	Comprehensive architecture and implementation proposal, version 0.1
Prepared for	Benjamin Goertzel and BGI Labs
Date	30 July 2026
Primary audience	Product leadership, system architects, Hyperon/OmegaClaw engineers, Buzz contributors, Scout contributors, security reviewers, and coding-agent teams

This proposal distinguishes clearly between capabilities already present in the public Buzz and Scout repositories and capabilities proposed for OmegaBuzz. Public repository status was reviewed as of 30 July 2026.

Document Purpose and Source Basis

This document proposes a coherent product and technical architecture for OMEGABUZZ. It is a synthesis of four ingredients:

1. the public Buzz collaboration platform and its Nostr-based signed event substrate;
2. the public Scout provenance-first archive reader and context-bundle workflow;
3. the OmegaHive organizational architecture, including explicit task ownership, shared and private AtomSpaces, deliberation rooms, governance, and evaluation;
4. Hyperon, MeTTa, AtomSpace, and PLN as the semantic memory and reasoning substrate for Omega-Claw agents and hives.

The public Buzz repository already provides a substantial collaboration surface: communities, channels, threads, direct messages, canvases, media, full-text search, workflows, signed Git events, an agent-oriented CLI, an ACP harness, and a hash-chain audit log. Buzz’s public design treats people and agents as first-class participants with cryptographic identities and a shared signed event history. Its own documentation also identifies implementation gaps that matter for production OmegaHive use, including absent enforced rate limiting, broad NIP-42 scopes, and incomplete end-to-end workflow approval resumption.[\[1, 2, 3\]](#)

Scout already demonstrates a complementary and unusually valuable interaction model: a deterministic read-only reader over a user-controlled substrate, exact source-range navigation, concepts and segments, a Library Card for selecting context, and Markdown or JSON bundles for human and agent use. Its public license file, however, currently reserves attribution and states that license terms remain to be determined by the copyright holder; redistribution or code incorporation therefore requires explicit clarification.[\[7, 8, 9\]](#)

The OmegaHive source documents supply the organizational discipline that neither product alone provides: a two-tier communication model, explicit task ownership, provenance, shared and individual AtomSpaces, a reviewed write path, wake-based deliberative seats, separately governed Resident Learners, permission enforcement outside prompts, deterministic metrics, and human-only recovery.[\[10, 11, 12, 13\]](#)

This is a design proposal, not a claim that every described capability is already implemented. The implementation plan deliberately separates an initial vertical slice from later federation, resident-learning, and high-assurance features.

Contents

Document Purpose and Source Basis	i
1 Executive Summary	1
1.1 The idea in one sentence	1
1.2 Why this combination matters	1
1.3 The five core product outcomes	2
1.4 The central rigor: four independent dimensions	2
1.5 What the user sees	3
1.6 The first useful release	3
2 Product Thesis, Scope, and Design Principles	4
2.1 Product thesis	4
2.2 Primary goals	4
2.3 Non-goals for the first release	5
2.4 Design principles	5
2.5 Architectural invariants	6
3 Technology Baseline and Integration Strategy	7
3.1 Buzz as the collaboration and signed-event substrate	7
3.1.1 Current Buzz gaps that OmegaBuzz must close	7
3.2 Scout as the provenance-first archive reader	7
3.3 OmegaHive as the organizational kernel	8
3.4 AtomSpace, MeTTa, and PLN as the semantic plane	8
3.5 Recommended codebase relationship	8
3.6 Licensing and provenance note	9
4 Overall Architecture	10
4.1 Layered system view	10
4.2 One history, several authoritative domains	10
4.3 Logical service decomposition	11
4.4 Consistency and failure semantics	12

5	Organizational Object Model	14
5.1	Overview	14
5.2	Community	14
5.3	Hive	14
5.4	Seat, session, and worker	15
5.5	Project	16
5.6	Epic	16
5.6.1	EpicFrame	17
5.7	Work order	18
5.8	Deliberation	19
6	Signed Event and State Architecture	20
6.1	Domain event envelope	20
6.2	Event families	20
6.3	Definition events versus transition events	21
6.4	Kind allocation strategy	21
6.5	Query tags	22
6.6	Event pipeline	22
6.7	Snapshots, replay, and repair	22
7	Layered AtomSpace and Semantic Memory Architecture	24
7.1	Why one global mutable AtomSpace is the wrong abstraction	24
7.2	Memory-space classes	25
7.3	Separate proposition identity from claim identity	25
7.4	Memory promotion	27
7.5	Working-memory dynamics	27
7.6	PLN and uncertain inference	27
7.7	Semantic-space access	28
8	Scout, Semantic Navigation, and Context Compilation	29
8.1	Architectural role	29
8.2	Archive Projector	29
8.3	Stable source anchors	29
8.4	Hybrid Context Broker	30
8.5	Context Cart and Context Bundles	30
8.6	Human questions Scout should answer	31
8.7	Agent-facing interface	31

9 Rooms, Deliberation, and Agent Participation	33
9.1 Typed room modes	33
9.2 The room authority banner	34
9.3 Deliberation protocol	34
9.4 Wake-based seats	35
9.5 Resident Learners	35
9.6 Attention and egress governance	36
9.7 Two-tier communication	36
9.8 Synthesis as a first-class capability	37
10 Minimalist User Experience and the Omega Agent Dashboard	38
10.1 UX thesis	38
10.2 Three primary modes	38
10.3 Global shell	38
10.4 Omega Agent Dashboard	39
10.4.1 Dashboard modules	39
10.5 Agent setup and coordination	40
10.6 Agent detail view	40
10.7 Hive view	41
10.8 Epic view	41
10.9 Room view	41
10.10 Knowledge and claim view	42
10.11 Decision and action inbox	42
10.12 Key user journeys	42
10.12.1 Journey A: create and launch a hive	43
10.12.2 Journey B: move from conversation to durable work	43
10.12.3 Journey C: launch into Scout and return with context	43
10.13 Accessibility and cognitive load	43
11 Networks of Hives and Knowledge Federation	44
11.1 Several hives in one community	44
11.2 Why cross-community federation must be explicit	44
11.3 Federation Gateway	44
11.4 Federated query versus replicated claim	45
11.5 Ontology and namespace management	45
11.6 Collective knowledge without monoculture	46
11.7 Cross-hive epics	46

12 Governance, Security, Privacy, and Safety	47
12.1 Authority is an intersection	47
12.2 Identity and key custody	47
12.3 Permission tiers and autonomy states	48
12.4 Typed ActionManifest	48
12.5 Policy and Action Broker	49
12.6 Knowledge compartments	49
12.7 Untrusted content and prompt injection	49
12.8 Search, embeddings, and derived-view leakage	50
12.9 Resident Learner privacy	50
12.10 Retention and deletion	51
12.11 Audit and non-repudiation	51
12.12 Human-only recovery	51
13 Reference Deployment and Operations	53
13.1 Single-node experimental deployment	53
13.2 Production trust zones	53
13.3 Container and service layout	54
13.4 Model routing and capability state	54
13.5 Observability	54
13.6 Backups and disaster recovery	55
13.7 Cost control	55
14 Metrics, Evaluation, and Experimental Program	56
14.1 Evaluation philosophy	56
14.2 Operational metrics	56
14.3 Semantic memory metrics	56
14.4 Scout and context metrics	57
14.5 Resident Learner metrics	57
14.6 Federation metrics	57
14.7 Core experiments	58
14.8 Staged Resident Learner experiment	59
14.9 Acceptance gates for the first pilot	59

15 Product and Technical Roadmap	60
15.1 Roadmap philosophy	60
15.2 Phase 0: contracts and Buzz hardening	60
15.3 Phase 1: single-hive organizational MVP	60
15.4 Phase 2: AtomSpace and Scout context loop	61
15.5 Phase 3: dashboard depth and Resident Learners	61
15.6 Phase 4: several hives in one community	61
15.7 Phase 5: cross-community federation	62
15.8 First vertical slice	62
16 Risks and Open Design Questions	63
16.1 Major risks and mitigations	63
16.2 Open questions requiring experiments	64
17 Conclusion	65
A Canonical Data-Schema Sketches	66
A.1 Seat descriptor	66
A.2 Epic descriptor	66
A.3 Work order	67
A.4 Deliberation descriptor	67
A.5 Claim	67
A.6 Memory promotion request	68
A.7 ActionManifest	68
B Service APIs and Events	70
B.1 Representative service interfaces	70
B.2 API requirements	70
C Guidance for Coding Agents	72
C.1 Development strategy	72
C.2 Recommended repository structure	72
C.3 Recommended coding-agent roles	74
C.4 Work-packet contract	75
C.5 Branching and review workflow	75
C.6 Definition of done	76
C.7 Continuous integration gates	76
C.8 Milestone-specific coding-agent guidance	77

C.8.1	Milestone M0: baseline and contracts	77
C.8.2	Milestone M1: Buzz hardening	77
C.8.3	Milestone M2: organizational vertical slice	77
C.8.4	Milestone M3: semantic memory and Scout	77
C.8.5	Milestone M4: Resident Learner shadow mode	77
C.8.6	Milestone M5: multi-hive and federation	77
C.9	Agent performance management and recovery	78
D	Initial Engineering Backlog	79
E	Testing Matrix	83
E.1	Test levels	83
E.2	Critical adversarial fixtures	84
F	Glossary	85
G	References	86

List of Figures

1.1	OmegaBuzz unifies four technical planes behind one human control surface.	1
1.2	Four dimensions that must never be collapsed into one confidence or trust score.	3
3.1	OmegaBuzz should extend Buzz without needlessly entangling every new service with the relay core.	9
4.1	Layered OmegaBuzz architecture. Human authority remains above the system; bounded workers remain below explicit policy and capability gates.	10
4.2	The event history anchors provenance; specialized stores support domain semantics; views remain disposable.	11
4.3	Durable command processing and recovery semantics.	13
5.1	Core OmegaBuzz organizational objects and their primary relationships.	14
5.2	A durable seat may have many sessions over time; each session may delegate bounded work.	16
5.3	An epic is a semantic and organizational context; the EpicFrame is its compact reviewable projection.	18
5.4	Recommended work-order lifecycle. Every transition is a typed event with actor and reason.	18
5.5	Deliberation lifecycle: conclude into committed objects or abandon explicitly.	19
6.1	OmegaBuzz event pipeline layered on the Buzz relay model.	22
7.1	Layered semantic memory. Promotion is explicit; federation is source-preserving.	24
7.2	A shared proposition may have many source-specific claims. This preserves disagreement and provenance.	26
7.3	Memory promotion is a reviewed semantic write path.	27
8.1	Scout remains a deterministic reader; ingestion and semantic enrichment are separate. . .	29
8.2	Hybrid semantic navigation with authorization after retrieval, not merely before it. . . .	30
8.3	Context compilation closes the loop from archive to agent work and back to sources. . . .	31
9.1	Different room types feed different kinds of state. A room badge should make this visible.	34
9.2	Resident Learner governance strengthens from observation toward organizational commitment.	35
9.3	Fast working traffic is curated out of rooms, not hidden from humans.	37
10.1	High-level wireframe for the Omega Agent Dashboard. It emphasizes attention, current intent, fleet state, and source-linked changes.	39
10.2	Minimal seat setup workflow. Advanced settings remain available by progressive disclosure.	40
10.3	Human-centered hive launch sequence.	43

10.4 Conversation-to-commit user journey.	43
10.5 Scout is not a separate archive island; it participates in the ongoing work loop.	43
11.1 Multiple hives may coexist in one community without merging internal state.	44
11.2 Cross-community knowledge flow preserves source identity and local admission.	44
12.1 Consequential authority is the intersection of independently enforced conditions.	47
12.2 Governed action flow. General reasoning agents propose; bounded workers execute.	49
12.3 Illustrative purpose-based knowledge compartments. No agent receives universal access by default.	49
13.1 Reference production trust zones. High-risk workers remain ephemeral and separately credentialed.	53
15.1 OmegaBuzz phased implementation. Each phase has a concrete exit gate.	60
15.2 The first vertical slice should exercise every central boundary once.	62
C.1 Coding-agent implementation and review flow.	75
E.1 OmegaBuzz test pyramid. Security and human factors are explicit levels, not afterthoughts.	83

List of Tables

1.1	Core outcomes	2
2.1	Core design principles	5
4.1	Canonicity matrix	11
4.2	Core OmegaBuzz services	11
6.1	Initial OmegaBuzz domain-event families	20
7.1	Memory-space classes	25
9.1	Recommended OmegaBuzz room types	33
12.1	Illustrative permission tiers	48
14.1	Operational and organizational metrics	56
14.2	Recommended core experiments	58
16.1	Major OmegaBuzz risks	63
B.1	Representative OmegaBuzz APIs	70
C.1	Coding-agent roles	74
D.1	Initial OmegaBuzz engineering backlog	79
E.1	Testing matrix	83

Executive Summary

1.1 The idea in one sentence

One sentence

OMEGABUZZ is a signed, governed, semantically navigable workspace in which humans and AI agents can converse, organize work, share structured memories, carry out bounded actions, and trace every important conclusion back to its source.

A simple way to understand the architecture is:

- **Buzz** supplies the place and the signed history.
- **OmegaHive** supplies the institution and operating discipline.
- **AtomSpace** supplies the layered semantic mind.
- **Scout** supplies the map, archive, and context compiler.
- The **Omega Dashboard** supplies the human control surface.

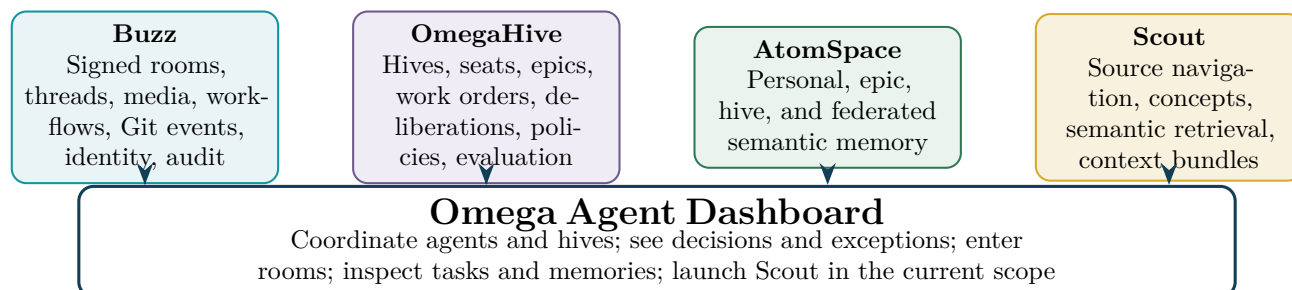


Figure 1.1: OmegaBuzz unifies four technical planes behind one human control surface.

1.2 Why this combination matters

Today, agent systems usually force users to choose among several incomplete patterns:

- a chatbot with no durable organizational state;
- a task runner with weak conversational context;
- a collaboration system with no semantic memory;
- a vector-memory layer with weak provenance and authority semantics;
- a multi-agent framework whose internal coordination is difficult for humans to inspect;
- or a powerful agent with dangerously broad credentials and unclear accountability.

OMEGABUZZ is intended to combine the missing pieces without collapsing them into one opaque “super-agent.” The architecture keeps identity, access, knowledge, and authority distinct; lets chat remain fluid while durable state remains explicit; and lets agents learn from broad experience without automatically turning that experience into organizational truth.

1.3 The five core product outcomes

Table 1.1: Core outcomes

Outcome	What the user experiences	Architectural basis
Remember	Agents and teams retain decisions, evidence, goals, concepts, and work history across months or years.	Layered AtomSpaces, signed events, Scout archive, content-addressed artifacts.
Coordinate	Work has owners, dependencies, current state, rooms, and explicit handoffs across many agents and hives.	Hives, seats, epics, work orders, deliberations, and task-state events.
Govern	Permissions, budgets, approvals, actions, memory promotion, and data access are enforced outside prompts.	Policy and Action Broker, Key and Capability Broker, typed manifests, compartment policy.
Understand	Humans and agents can answer “why” by opening exact messages, evidence, derivations, and prior decisions.	Scout source anchors, claim/evidence graph, derivation traces, Context Bundles.
Improve	The organization can measure learning, coordination, cost, reliability, and delegation, then revise its agents and policies.	Deterministic telemetry, retrospectives, Resident Learner experiments, replayable events.

1.4 The central rigor: four independent dimensions

Every important object in OMEGABUZZ should expose four different answers:

1. **Authorship:** who produced or signed it?
2. **Visibility:** who may see or use it, and for what purpose?
3. **Epistemic standing:** is it observed, inferred, disputed, stale, or supported by evidence?
4. **Organizational authority:** is it merely conversational, proposed, reviewed, or committed?

A signed message proves authorship, not truth. A high-confidence claim does not authorize an action. Room membership does not imply production access. Conversational consensus does not create a committed organizational fact.

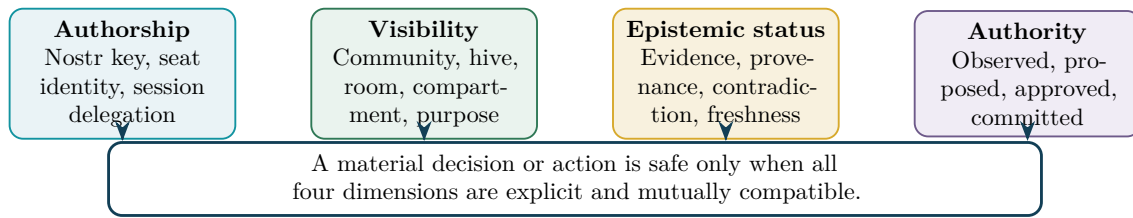


Figure 1.2: Four dimensions that must never be collapsed into one confidence or trust score.

1.5 What the user sees

The minimalist user experience has three primary modes:

Coordinate

The Omega Agent Dashboard: hives, agents, active epics, decisions requiring attention, blocked work, budgets, health, permissions, and unusual activity.

Converse Buzz rooms: ordinary channels, epic rooms, bounded deliberations, workcells, reviews, nurseries, and cross-hive rooms.

Explore Scout: provenance-first search and semantic navigation over conversations, claims, artifacts, tasks, decisions, and AtomSpace concepts, with context-bundle compilation for humans and agents.

The dashboard is not intended to visualize every message or atom. It is an **exception-first control surface**: show the human what needs a decision, what is blocked, what changed, what is expensive, what is risky, and what has become epistemically uncertain. Everything else remains available by drill-down.

1.6 The first useful release

The first vertical slice should prove one complete loop rather than implement the whole vision:

1. a human creates one hive and two seats;
2. the human opens an epic and a bounded deliberation room;
3. two agents participate with separate identities and memories;
4. a synthesizer closes the deliberation into a committed decision and work order;
5. a candidate claim enters an epic AtomSpace, is reviewed, and is promoted into hive memory;
6. Scout can trace the claim and decision back to exact signed room events;
7. the dashboard shows the resulting work, memory, provenance, cost, and outstanding decisions.

This single flow tests the architectural heart of OMEGABUZZ: signed identity, bounded conversation, explicit work, semantic memory, reviewed promotion, and source-traceable navigation.

Product Thesis, Scope, and Design Principles

2.1 Product thesis

The durable asset in a human-agent organization is not the roster of models currently producing text. Models, prompts, and worker processes will change rapidly. The compounding asset is the organization's governed history: its goals, concepts, decisions, tasks, evidence, relationships, learned procedures, delegation boundaries, and source-linked semantic memory.

Product thesis

OMEGABUZZ should make the governed, provenance-complete cognitive state of a human-agent organization more valuable over time, while keeping its models and execution workers replaceable.

This aligns with the broader Hyperon framing that an intelligent system should remember, coordinate, govern, and improve through persistent metagraph memory, explicit reasoning programs, uncertain inference, and bounded self-modification rather than relying only on the transient internal state of one foundation model.^[14]

2.2 Primary goals

1. Give humans and agents a shared collaboration surface with first-class, cryptographically attributable identities.
2. Make hives, seats, sessions, workers, epics, work orders, deliberations, decisions, artifacts, and claims explicit objects rather than conventions embedded in prose.
3. Support layered semantic memory: private experiential memory, small-group working memory, durable hive memory, and federated collective knowledge.
4. Preserve exact provenance from summaries and claims back to messages, files, evidence, tools, model routes, and prior decisions.
5. Separate conversational freedom from organizational authority.
6. Enforce permissions, action scope, budgets, and credentials outside model prompts.
7. Give users one minimalist control surface for a potentially large fleet of agents and hives.
8. Support wake-based deliberative seats and separately governed Resident Learners.
9. Enable networks of hives to collaborate and exchange knowledge without silently merging authority or private state.
10. Be incrementally implementable on top of the public Buzz and Scout codebases.

2.3 Non-goals for the first release

The first release is not:

- one universal global AtomSpace into which every agent can write;
- an unrestricted autonomous organization;
- a claim that chat transcripts are organizational truth;
- a replacement for deterministic task, policy, credential, or artifact systems;
- a vector database treated as the source of truth;
- automatic cross-relay replication of private agent state;
- full semantic understanding of every historical message;
- a detailed visual IDE for every AtomSpace relation;
- an attempt to force every agent to be continuously resident;
- or a deep permanent fork that diverges irreversibly from upstream Buzz.

2.4 Design principles

Table 2.1: Core design principles

Principle	Meaning
Signed history, typed state	Conversation may be free-form, but durable transitions use signed, schema-versioned domain events.
Chat is not the write path	Rooms support discussion and authorization; committed objects carry durable organizational state.
One history, specialized stores	The event ledger is canonical for signed history; object storage, AtomSpace, task state, and indexes have explicit domain roles.
Provenance before fluency	A useful answer includes receipts and source anchors; polished prose without traceability is not durable knowledge.
Epics are fuzzy, ownership is crisp	An epic may loosely gather related work; each executable work order still has an explicit owner, state, budget, and acceptance test.
Private learning, reviewed promotion	An agent may update provisional private memory freely within policy; shared durable memory requires a promotion path.
Authority outside prompts	Credentials, routes, scopes, policies, manifests, and system health determine what can happen.
Replaceable agents, durable institution	Seats, memories, event history, artifacts, and governance outlive individual sessions and model providers.
Federate by assertion, not overwrite	A foreign hive’s claim remains a sourced foreign assertion until locally admitted.
Progressive disclosure	The UI shows decisions and exceptions first, then permits source-level drill-down.

Principle	Meaning
Shadow before enforcement	Governors, filters, semantic suppression, and autonomy policies are evaluated in shadow mode before they block behavior.
Human-only recovery	A malfunction of every agent must not lock humans out of the system.

2.5 Architectural invariants

The following invariants should be machine-tested wherever possible:

1. Nothing said in a room becomes organizationally real solely because it was said.
2. Every durable organizational mutation has a signed typed event and an authorization basis.
3. Every durable semantic assertion is provenance-backed or explicitly labeled as judgment, hypothesis, or imported assertion.
4. Authorship, visibility, epistemic status, and authority remain separate fields.
5. Private or working memory cannot silently confer organizational authority.
6. Fuzzy epic membership never creates fuzzy task ownership.
7. No imported cross-hive claim becomes a local fact without local admission.
8. Search indexes, embeddings, summaries, and Scout views are rebuildable projections, not sources of truth.
9. No general LLM process holds unrestricted long-lived signing keys or production credentials.
10. A committed claim is superseded or retracted; its history is not overwritten.
11. Every consequential action is independently validated and linked to its work order, manifest, authority, and result.
12. High-volume machine traces remain inspectable without overwhelming human-facing rooms.

Technology Baseline and Integration Strategy

3.1 Buzz as the collaboration and signed-event substrate

Buzz is a strong starting point because it already embodies several choices that OmegaHive needs:

- humans and agents participate in the same workspace with cryptographic identities;
- messages, reactions, workflow steps, reviews, and Git events share a signed Nostr event shape;
- communities are bounded by the relay URL or host;
- the relay verifies signatures, enforces channel membership, persists events, fans them out, indexes eligible content, and triggers workflows;
- the implementation provides a desktop application, agent-oriented CLI, ACP harness, Git integration, full-text search, object storage integration, and a hash-chain audit log.[1, 2, 3, 6]

Buzz’s public project vision also makes an important distinction useful for OmegaBuzz: a project may group resources without gaining authority over them. OmegaBuzz generalizes this into the epic model, where semantic relatedness does not silently grant operational control.[4]

3.1.1 Current Buzz gaps that OmegaBuzz must close

As of 30 July 2026, the public Buzz architecture document explicitly lists several relevant gaps:

- NIP-42 authentication currently grants all known scopes in its authentication context;
- rate-limit configuration exists but no production rate limiter is enforced;
- workflow approval suspension and resumption are not wired end to end;
- several workflow actions remain stubbed;
- the built-in audit action set is narrower than the organizational, memory, authorization, and federation audit required by OmegaBuzz.[3]

These are not reasons to reject Buzz. They are a clearly bounded upstream/downstream hardening agenda.

3.2 Scout as the provenance-first archive reader

Scout’s strongest idea is not simply “semantic search.” It is the separation of a deterministic read-only reader from ingestion and model-driven enrichment. The current repository reads a plain folder substrate, exposes workspaces, sources, segments, concepts, exact source ranges, lifecycle overlays, a Library Card,

and Markdown or JSON context bundles. It calls no LLM at query time and keeps source inspection primary.[7, 8]

OmegaBuzz should preserve this property. The proposed semantic Context Broker should be a separate service that produces authorized candidate results and links; Scout remains the human-auditable reader and context assembler.

3.3 OmegaHive as the organizational kernel

The OmegaHive design contributes the organizational semantics absent from a collaboration platform alone:

- fast machine working traffic plus a curated human-legible layer;
- explicit task ownership and persistent work state;
- shared and individual AtomSpaces with provenance;
- a reviewed write path for durable conclusions;
- a version-controlled constitution and permission policy;
- deterministic quantitative metrics plus qualitative retrospective review;
- wake-based seats, bounded workers, deliberation rooms, and separately governed Resident Learners;
- fleet management, versioned prompts and policies, and a human-only recovery path.[10, 11, 12]

3.4 AtomSpace, MeTTa, and PLN as the semantic plane

AtomSpace provides a typed metagraph capable of representing n-ary relationships, contexts, time, provenance, claims about claims, evidence links, executable rules, and uncertain truth values in one substrate. MeTTa provides programmable querying and transformation. PLN can perform uncertain, revisable inference while retaining explicit support, contradiction, source dependence, and confidence semantics.[13, 14]

OmegaBuzz does not require every feature to be expressed in MeTTa or PLN. Deterministic policy, relational task state, signed event handling, object storage, and authorization remain ordinary engineered services. AtomSpace is used where semantic relationships and revisable inference provide real value.

3.5 Recommended codebase relationship

Build OmegaBuzz as a downstream distribution and extension layer around upstream Buzz, with most OmegaHive, AtomSpace, Scout-projector, policy, and federation functions implemented as sidecar services and extension crates. Maintain a small, reviewable patch stack for generic Buzz improvements and upstream those improvements whenever practical.

This approach balances three needs:

1. OmegaBuzz needs desktop and room UI changes that will initially require a Buzz fork or downstream branch.
2. The signed event protocol lets many OmegaBuzz capabilities remain loosely coupled sidecars.
3. Generic hardening such as scoped authentication, production rate limiting, approval resumption, and extension-kind registration benefits upstream Buzz and should not remain a private divergence.

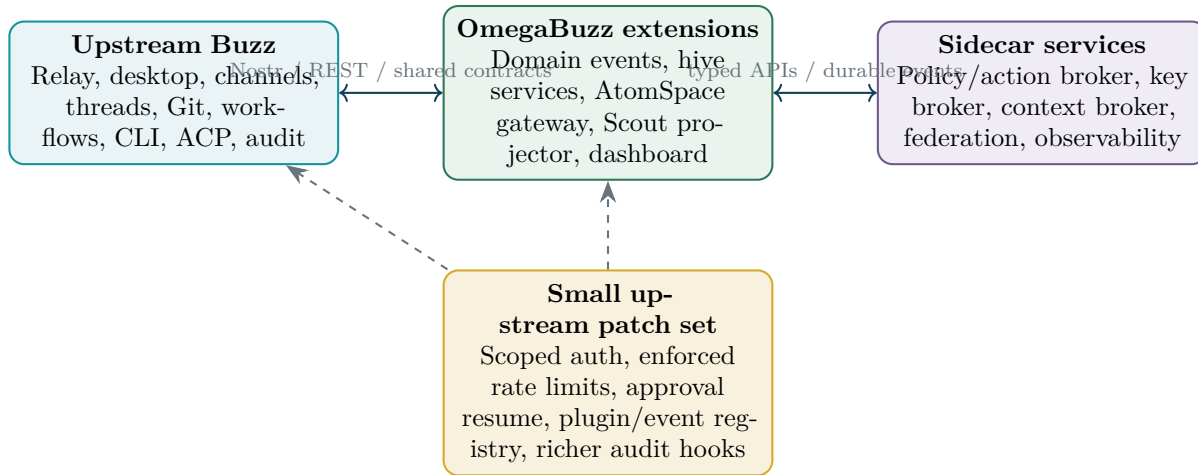


Figure 3.1: OmegaBuzz should extend Buzz without needlessly entangling every new service with the relay core.

3.6 Licensing and provenance note

Buzz is published under Apache 2.0. Scout's current public license file identifies the SingularityNET Foundation as copyright holder, requires attribution, and states that license terms remain to be determined. Before Scout code is redistributed inside an OmegaBuzz product or combined work, BGI Labs should obtain an explicit license or arrange a clear relicense. The design concepts can still be implemented independently, but provenance and attribution should remain explicit.[2, 9]

Overall Architecture

4.1 Layered system view

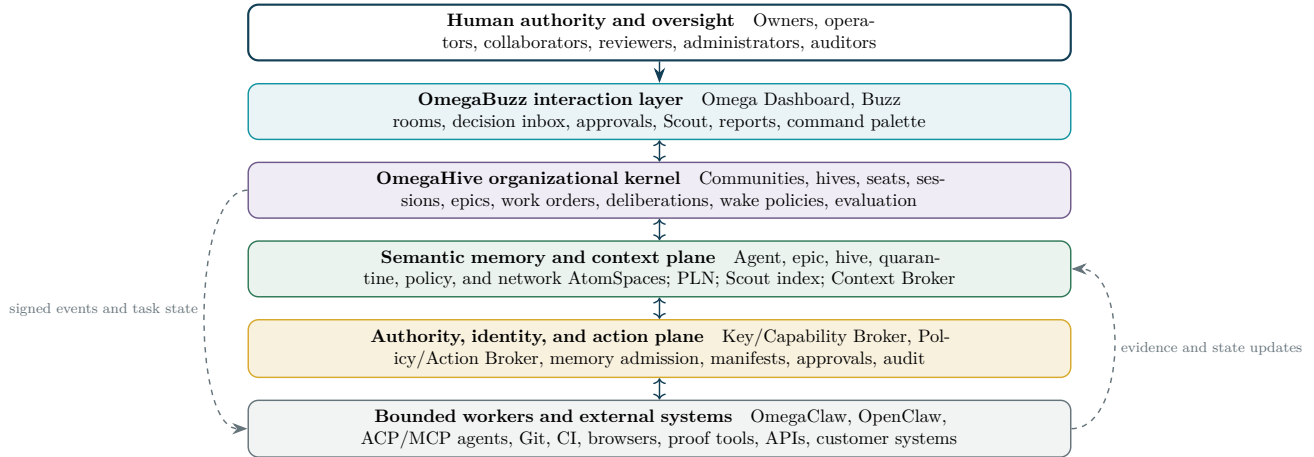


Figure 4.1: Layered OmegaBuzz architecture. Human authority remains above the system; bounded workers remain below explicit policy and capability gates.

4.2 One history, several authoritative domains

It is tempting to declare one database the universal source of truth. That is too crude. OmegaBuzz should instead define a **canonical source by information class**, while maintaining one signed history of durable organizational changes.

Table 4.1: Canonicity matrix

Information class	Canonical source	Rebuildable projections or views
Room messages, authorship, reactions	Buzz signed event log	Search index, Scout archive pages, digests
Hive and organizational transitions	Signed OmegaBuzz domain events	Dashboard, task board, epic views, retrospectives
Large files and artifacts	Content-addressed object store or Git	Buzz cards, Scout source pages, semantic metadata
Private experiential memory	Agent-local experience store / AtomSpace	Summaries, selected promoted claims
Shared working memory	Epic or workcell AtomSpace with write provenance	Context bundles, salience views, current EpicFrame
Durable hive knowledge	Admitted knowledge events plus hive AtomSpace state	Semantic indexes, Scout concept views, embeddings
Actions and approvals	Immutable ActionManifest and result events	Decision inbox, action dashboard, audit reports
Policies and delegations	Versioned policy store and signed grant events	UI badges, AtomSpace policy views, effective-authority calculations
Search and embeddings	No independent authority	Rebuilt from currently authorized source content



Figure 4.2: The event history anchors provenance; specialized stores support domain semantics; views remain disposable.

4.3 Logical service decomposition

Table 4.2: Core OmegaBuzz services

Service	Primary responsibility
Buzz Relay and Clients	Existing signed event transport, communities, channels, threads, media, Git, workflows, full-text search, and desktop/mobile surfaces.
Hive Registry	Hive definitions, charters, ownership, namespaces, policies, and lifecycle.
Seat and Session Manager	Durable seat identity, memory lineage, session activation, short-lived signing delegation, wake/sleep state, and model route.

Service	Primary responsibility
Epic and Task Service	Epics, EpicFrames, work orders, ownership, dependencies, status, budgets, acceptance, and escalation.
Deliberation Service	Open/close lifecycle, roster, chair/wake policy, synthesizer, owning epic, archival, landing-rate metrics.
Wake Scheduler / Conversation Governor	Wake-based turns, Resident Learner attention budgets, response leases, loop breakers, quiet/pause controls.
AtomSpace Gateway	Namespace-aware access to private, epic, hive, quarantine, policy, and network semantic spaces.
Memory Admission Service	Schema validation, provenance checks, contradiction handling, promotion, supersession, and retraction.
Archive Projector	Converts authorized Buzz events, artifacts, tasks, claims, and decisions into a Scout-compatible substrate with stable source anchors.
Context Broker	Combines full-text, vector, graph, temporal, and task-state retrieval under authorization; creates Context Bundles.
Key and Capability Broker	Protects long-lived keys, issues session-scoped sign grants and task-specific capabilities, supports revocation.
Policy and Action Broker	Evaluates typed action manifests, approvals, delegation, route state, system health, rollback, and validation requirements.
Artifact Registry	Content hashes, metadata, classification, retention, provenance, and links to object storage or Git.
Federation Gateway	Signed knowledge export/import, ontology mapping, quarantine, local admission, expiry, and revocation.
Observability and Cost Service	Deterministic task, event, agent, model, memory, search, safety, cost, and quality metrics.

4.4 Consistency and failure semantics

OmegaBuzz should assume that projections and external services sometimes lag or fail. The system should make degradation visible rather than silently pretending to be consistent.

- Every durable command carries an idempotency key and produces one or more correlation-linked events.
- Projectors checkpoint by event ID and can replay from the ledger.
- A view shows its last successful projection time and stale status when relevant.
- Chat may remain available during a semantic-index outage, but memory promotion and consequential actions fail closed if their policy, audit, or authority dependencies are unavailable.
- AtomSpace snapshots accelerate recovery but do not replace event provenance.
- Partial model or worker outputs remain available as explicitly incomplete artifacts; they do not become successful task transitions.
- Retries are bounded, idempotent, and visible.

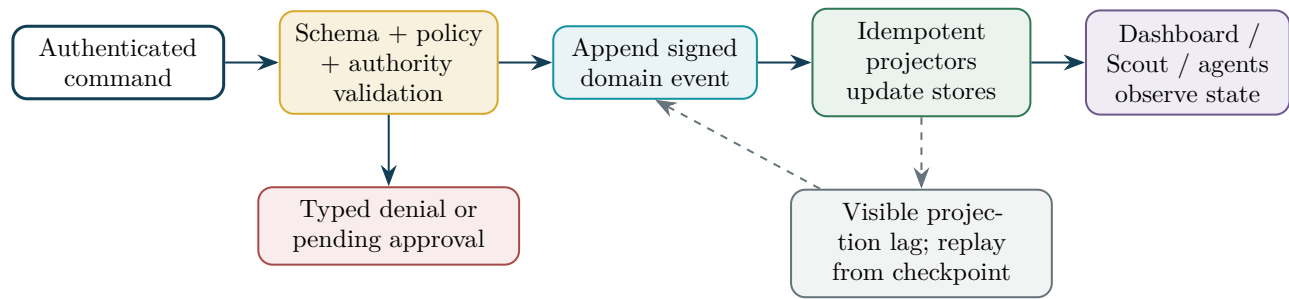


Figure 4.3: Durable command processing and recovery semantics.

Organizational Object Model

5.1 Overview

OmegaBuzz should represent organization explicitly. The goal is not to force human collaboration into rigid forms; it is to ensure that the parts with operational or epistemic consequences are not left implicit in chat.

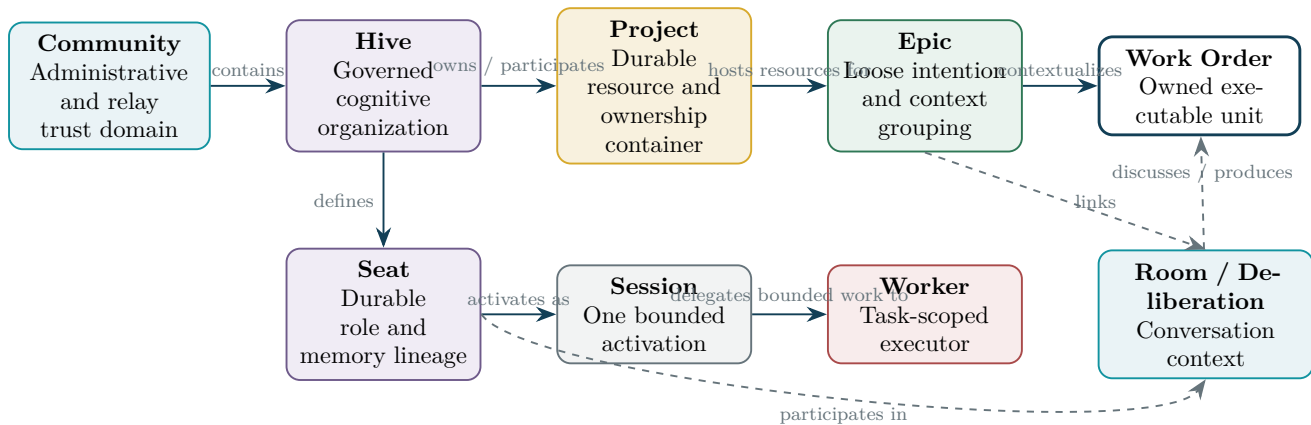


Figure 5.1: Core OmegaBuzz organizational objects and their primary relationships.

5.2 Community

A **Community** is the Buzz relay-facing administrative and security trust domain selected by a URL or host. It may contain one or several hives. Community boundaries govern tenant-visible profiles, rooms, direct messages, memberships, workflows, search, media metadata, Git state, and audit views. The same Nostr public key may join multiple communities, but state does not automatically inherit across hosts.[3, 6, 5]

A sensitive customer deployment may use one community per customer and one hive per community. A research environment may place several hives in one community and expose a shared commons.

5.3 Hive

A **Hive** is a persistent governed organizational and cognitive unit. It exists independently of whether any agent process is awake.

Listing 5.1: Illustrative Hive descriptor

```
{
  "hive_id": "hive-research-01",
  "name": "OmegaHive Research",
  "community_id": "community-bgi-labs",
```

```

"charter_ref": "artifact://hive-charter-v7",
"human_authorities": ["npub-human-owner"],
"seat_ids": ["seat-xirtus", "seat-macgyver", "seat-bossytron"],
"memory_spaces": ["as://hive/research", "as://hive/policy"],
"default_permission_policy": "policy://hive-research-v7",
"evaluation_policy": "policy://eval-research-v3",
"recovery_authority": ["human-owner", "human-sysadmin"],
"status": "active"
}

```

A hive owns or references:

- a charter and versioned constitution;
- human authorities and recovery owners;
- seats and worker classes;
- approved model routes and tools;
- private and shared memory spaces;
- budgets, permission tiers, and action policies;
- epics, tasks, deliberations, and artifacts;
- evaluation and retrospective policies;
- federation and disclosure rules.

5.4 Seat, session, and worker

These three concepts should never be conflated.

Seat	A durable role, identity, charter, memory lineage, operating protocol, social position, and public contribution history. Examples include Chief of Staff, Philosopher, Librarian, Assurance, or Cartographer.
Session	One bounded activation of a seat under a specific model route, context bundle, budget, signing delegation, and wake reason.
Worker	A narrow execution process assigned one work order or typed action manifest. A worker may be stateless except for task-local checkpoints and should normally have less authority than the seat that requested it.

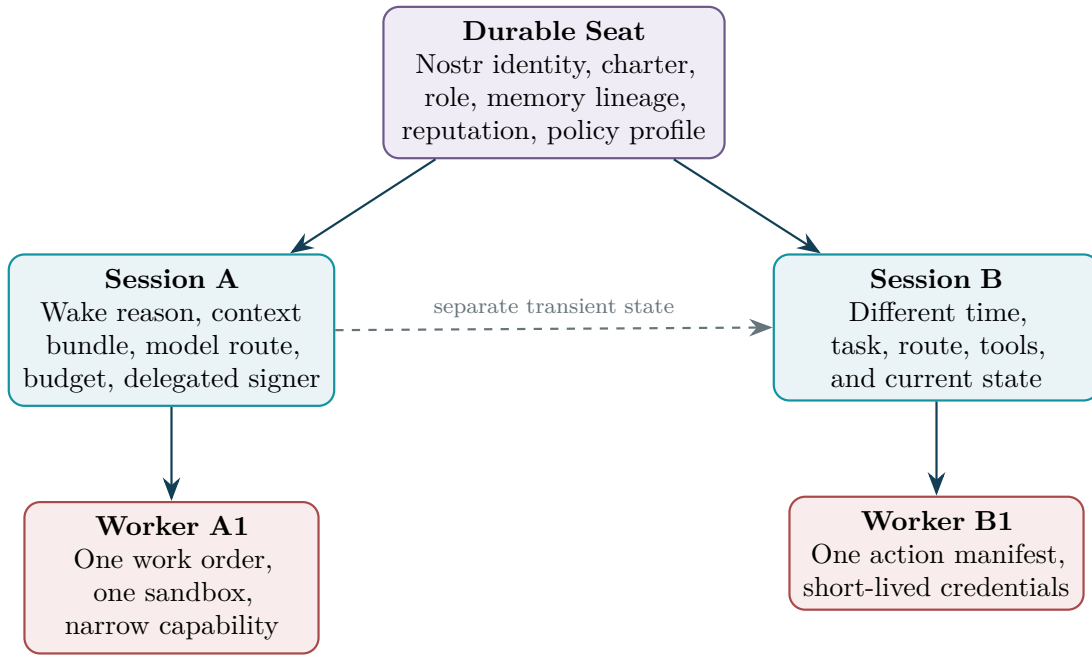


Figure 5.2: A durable seat may have many sessions over time; each session may delegate bounded work.

The persistent Buzz/Nostr identity should normally identify the seat. Session processes should use a short-lived delegated signer or signing service rather than receiving the seat’s long-term private key. Worker identities may be separate service identities with even narrower authorization.

5.5 Project

A **Project** is a durable resource and ownership container: repositories, documents, deliverables, stakeholders, policies, and long-term objectives. Buzz’s proposed multi-repository project model is a useful baseline because grouping does not transfer authority over the grouped repositories.[4]

A project may participate in many epics, and an epic may span several projects.

5.6 Epic

An **Epic** is a loosely bounded, evolving intention-context. It groups work that contributes to a broad purpose even when the exact task hierarchy is not yet known.

$$E = \langle id, intent, questions, scope, exclusions, stewards, participants, resources, status, synthesis, policies \rangle. \quad (5.1)$$

Membership should be represented as an assertion rather than a hard tree edge:

$$\text{ContributesTo}(x, E, r, w, p, [t_0, t_1]), \quad (5.2)$$

where r is the object’s role, w is optional semantic relevance, p is provenance, and $[t_0, t_1]$ is a validity interval.

This supports:

- one task contributing to several epics;
- one deliberation bridging two strategic concerns;
- retrospective discovery that an artifact matters elsewhere;
- AI-suggested epic membership that does not silently alter task state;
- and evolving scope without constant restructuring.

Epic membership may be fuzzy. Task ownership, authorization, completion criteria, and accountability may not be fuzzy.

5.6.1 EpicFrame

Every active epic should maintain a compact, reviewable **EpicFrame**:

- current intent and scope;
- key open questions;
- current synthesis and major alternatives;
- accepted decisions and minority objections;
- active, blocked, and unowned work;
- important artifacts, claims, and evidence gaps;
- participating hives and people;
- recent changes and next review date.

The EpicFrame is a rehydration aid and navigation summary, not a replacement for its sources. Each version links to the events and objects from which it was synthesized.

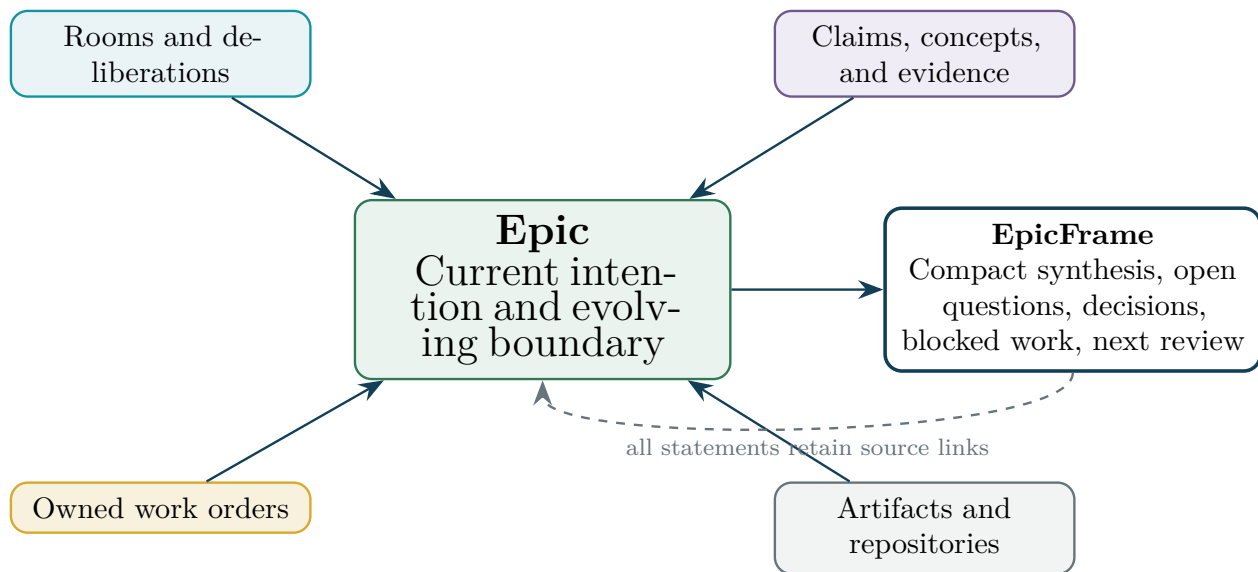


Figure 5.3: An epic is a semantic and organizational context; the EpicFrame is its compact reviewable projection.

5.7 Work order

A **WorkOrder** is the smallest organizationally owned unit of non-trivial work. It records:

- objective and expected output;
- requester, owner, reviewer, and owning hive;
- source epic or epics;
- inputs, dependencies, and blocking reasons;
- budget, tools, model-route requirements, and permission tier;
- acceptance tests and evidence of completion;
- due date, escalation policy, and provenance;
- status and supersession history.

Recommended state machine:

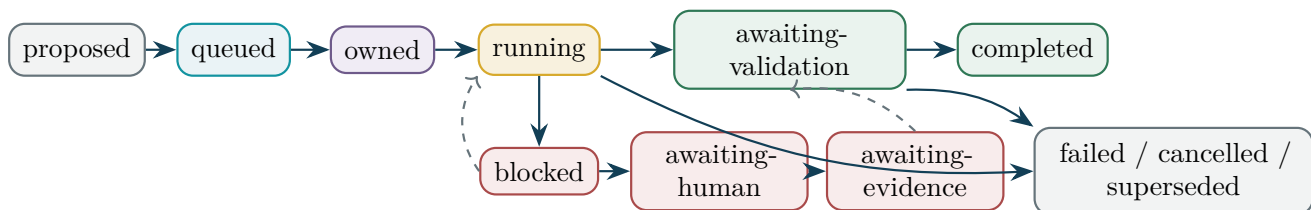


Figure 5.4: Recommended work-order lifecycle. Every transition is a typed event with actor and reason.

5.8 Deliberation

A **Deliberation** is a bounded conversational episode opened to answer a question or make a decision. It is neither an epic nor a task queue.

It opens with:

- a precise question or decision requirement;
- owning epic or project;
- declared roster;
- chair or wake policy;
- designated synthesizer;
- visibility and retention scope;
- deadline or close condition.

It closes with:

- committed decisions, claims, artifacts, and work orders;
- unresolved questions and preserved objections;
- source-linked transcript archive;
- or an explicit abandonment event.

The source OmegaHive deliberation design correctly separates chairing from synthesis and treats landing rate as a key metric: a room is useful only when its conclusions become committed state rather than dying in conversation.[11]

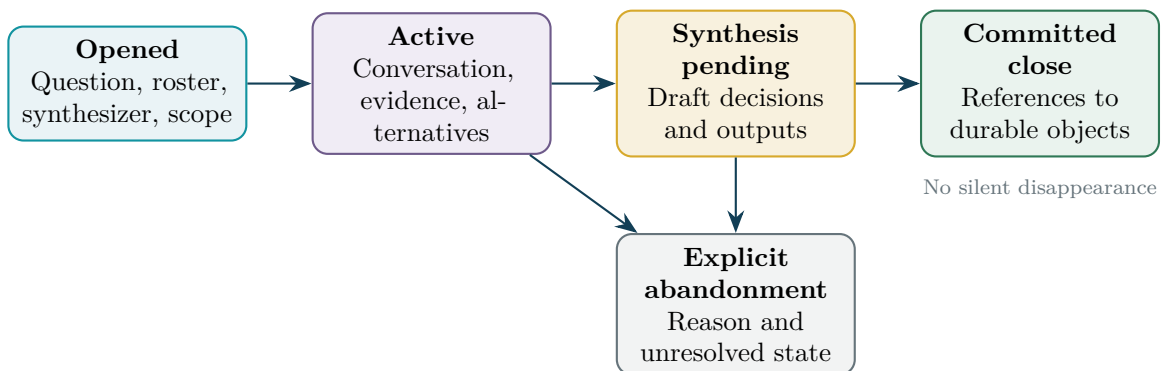


Figure 5.5: Deliberation lifecycle: conclude into committed objects or abandon explicitly.

Signed Event and State Architecture

6.1 Domain event envelope

Buzz already uses signed Nostr events as the common wire shape. OmegaBuzz should add typed organizational payloads and queryable tags rather than encoding state changes in prose.

Listing 6.1: Illustrative OmegaBuzz domain event

```
{
  "schema": "org.bgi.omegabuzz.event.v1",
  "type": "DeliberationClosed",
  "object_id": "delib-2026-07-30-memory-federation",
  "community_id": "community-bgi-labs",
  "hive_id": "hive-research-01",
  "epic_ids": ["epic-collective-memory"],
  "actor": {
    "principal": "npub...",
    "seat_id": "seat-synthesizer",
    "session_id": "session-98f2"
  },
  "occurred_at": "2026-07-30T21:14:00Z",
  "causation_ids": ["buzz-event-...", "work-..."],
  "correlation_id": "epic-collective-memory",
  "classification": "internal",
  "policy_version": "hive-policy-7",
  "payload": {
    "decision_refs": ["decision-..."],
    "work_order_refs": ["work-..."],
    "unresolved_refs": ["question-..."],
    "transcript_ref": "buzz://community/channel/event-range"
  }
}
```

The outer Nostr envelope supplies event ID, public key, kind, tags, content, and signature. The OmegaBuzz payload supplies domain semantics.

6.2 Event families

Table 6.1: Initial OmegaBuzz domain-event families

Family	Representative events
Hive and identity	HiveDefined, HiveCharterChanged, SeatDefined, SeatCharterChanged, SessionDelegated, SessionStarted, SessionEnded, SeatPaused.

Family	Representative events
Epics and tasks	EpicDefined, EpicMembershipAsserted, EpicMembershipRejected, EpicSynthesized, WorkOrderCreated, WorkOrderAssigned, WorkOrderTransitioned, WorkOrderSuperseded.
Deliberation	DeliberationOpened, TurnGranted, DeliberationSynthesisProposed, DeliberationClosed, DeliberationAbandoned.
Knowledge	MemoryProposed, MemoryPromoted, ClaimAdmitted, ClaimContradicted, ClaimSuperseded, ClaimRetracted, MemoryExpired.
Artifacts and decisions	ArtifactRegistered, ArtifactReviewed, ArtifactAccepted, DecisionProposed, DecisionCommitted, DecisionSuperseded.
Authority	CapabilityGranted, CapabilityRevoked, PermissionChanged, DelegationChanged, PolicyActivated, PolicyRetired.
Action	ActionManifestProposed, ActionApprovalRequired, ActionAuthorized, ActionStarted, ActionValidated, ActionFailed, ActionRolledBack.
Federation	KnowledgeExported, KnowledgeImported, ImportQuarantined, ImportAdmitted, ForeignClaimSuperseded, ExportRevoked.
Governance and health	BudgetExceeded, LoopBreakerTriggered, AgentMuted, CircuitBreakerActivated, RecoveryStarted, RetrospectiveCommitted.

6.3 Definition events versus transition events

OmegaBuzz should distinguish two event semantics:

Definition events

Replaceable or parameterized-replaceable descriptions of current objects, such as a hive definition, seat charter, epic descriptor, or room metadata.

Transition events

Immutable events recording that something happened, such as assignment, promotion, authorization, closure, revocation, failure, or supersession.

Definitions provide a convenient current view. Transitions provide history. A current definition should point to the transition events that justify its state.

6.4 Kind allocation strategy

The public Buzz repository uses standard Nostr kinds where available and a custom range for Buzz-specific features. OmegaBuzz should coordinate with upstream before assigning permanent numbers. A reasonable development strategy is:

- reserve a provisional private range, for example a conflict-checked slice of 47xxx;
- place the stable schema name and version in content and tags, so numbers remain dispatch identifiers rather than the sole semantics;

- use standard kinds for ordinary chat, reactions, profiles, NIP-29 groups, and NIP-34 Git events;
- use immutable custom kinds for domain transitions;
- use parameterized replaceable events for current definitions when that matches Nostr semantics;
- use ephemeral events only for presence, typing, transient turn grants, or other state that should not be replayed as durable history.

6.5 Query tags

Each domain event should include a small, stable set of query tags where relevant:

Listing 6.2: Illustrative Nostr tags for an OmegaBuzz event

```
[
  ["hive", "hive-research-01"],
  ["epic", "epic-collective-memory"],
  ["obj", "deliberation", "delib-2026-07-30-memory-federation"],
  ["schema", "org.bgi.omegabuzz.event.v1"],
  ["status", "closed"],
  ["e", "causation-event-id"],
  ["p", "relevant-participant-pubkey"]
]
```

Tags should make common relay queries efficient without leaking sensitive payload content into indexes. Sensitive classifications may require opaque IDs and authorization-aware resolution.

6.6 Event pipeline

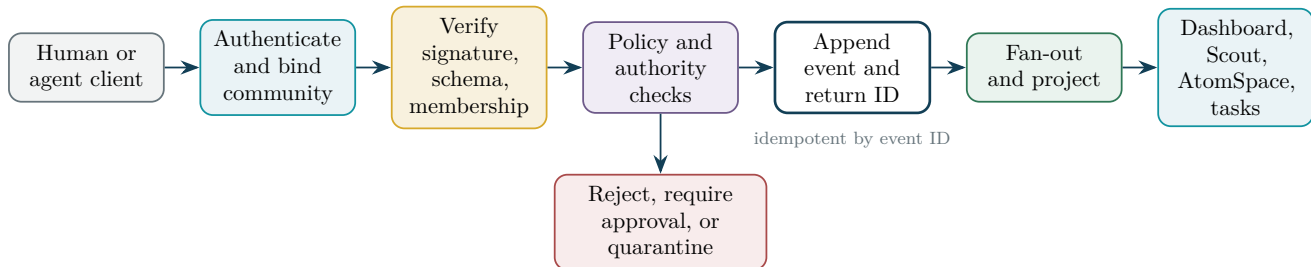


Figure 6.1: OmegaBuzz event pipeline layered on the Buzz relay model.

6.7 Snapshots, replay, and repair

The event log should support deterministic rebuild of all shared organizational and semantic projections. Because large AtomSpaces may be expensive to replay from genesis, the system should create signed snapshots that record:

- snapshot schema and software versions;
- last included event ID and ordering checkpoint;

- content hash and object-store location;
- namespaces and compartments included;
- known omissions and quarantined regions;
- signer and policy version;
- validation results against a replay sample.

Repair proceeds from the latest valid snapshot plus subsequent events. A snapshot never removes the need to retain the provenance references required by policy.

Layered AtomSpace and Semantic Memory Architecture

7.1 Why one global mutable AtomSpace is the wrong abstraction

A single shared graph for every agent and every hive would create uncontrolled cognitive noise, weak privacy, confusing authority, difficult rollback, and poor scaling. “Shared AtomSpace” should therefore mean a family of spaces with explicit scope, write policy, retention, and admission semantics.

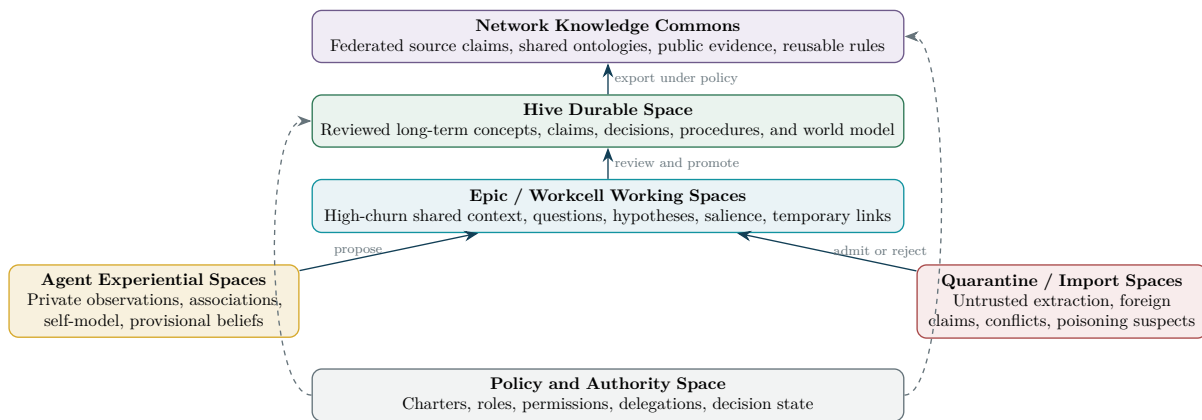


Figure 7.1: Layered semantic memory. Promotion is explicit; federation is source-preserving.

7.2 Memory-space classes

Table 7.1: Memory-space classes

Space	Purpose	Default writers	Authority and retention
Agent experiential	Personal observations, associations, self-model, provisional beliefs, episodic traces	One seat and its sessions	Private/provisional; source-linked; variable decay and retention
Epic working	Shared active context for a bounded group	Epic participants under write policy	Provisional; high churn; supports salience and TTL
Hive durable	Reviewed long-term semantic memory and world model	Memory Admission Service	Admitted hive knowledge; supersession rather than overwrite
Quarantine/import	Untrusted sources, extraction results, foreign claims, contradiction sets	Ingest and federation services	No authority until reviewed; stronger isolation
Network commons	Knowledge intentionally shared among hives	Federation and admission services	Source-scoped collective knowledge; no universal consensus
Policy/authority	Charters, permissions, delegations, decision state	Deterministic policy services and named authorities	Organizationally authoritative; tightly access-controlled

The Omega Purple security design illustrates why purpose-specific compartments are a general architectural need, not a cybersecurity oddity: different agents should see only the world-model views, plans, approvals, and evidence required by their role and task.[13]

7.3 Separate proposition identity from claim identity

Multiple agents or hives may assert the same proposition under different contexts, evidence, and time windows. OmegaBuzz should therefore represent a proposition once while representing each assertion as a separate claim object.

Listing 7.1: Illustrative MeTTa-style proposition and claim instances

```
(Proposition p-17
  (EvaluationLink
    (PredicateNode "UsefulFor")
    (ListLink system-X use-case-Y)))

(Claim c-101
  (content p-17)
  (context epic-42)
  (asserted-by seat-macgyver)
  (source buzz-event-abc))
```

```

(epistemic-status inferred)
(authority-status proposed)
(valid-from 2026-07-30)
(stv 0.78 0.62))

(Claim c-102
 (content p-17)
 (context hive-security)
 (asserted-by seat-purplebot)
 (source evidence-bundle-991)
 (epistemic-status observed-analog)
 (authority-status admitted)
 (stv 0.64 0.91))

```

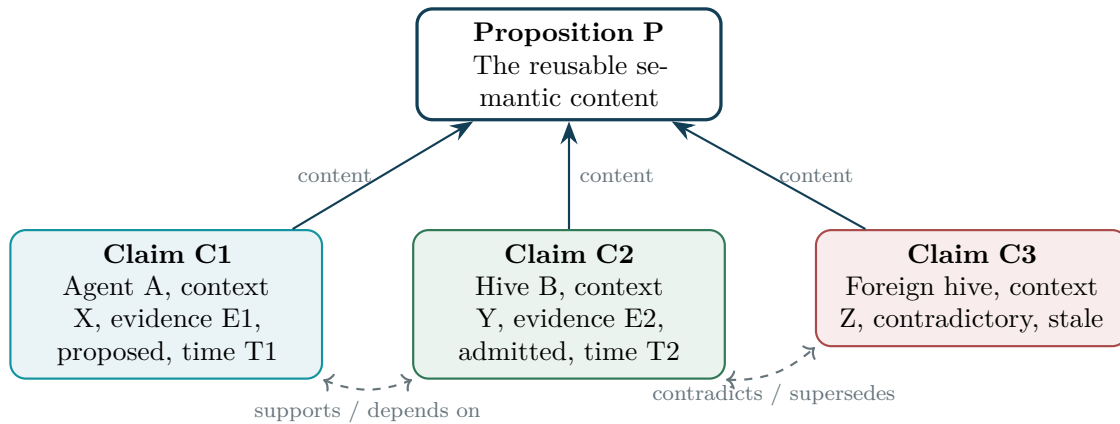


Figure 7.2: A shared proposition may have many source-specific claims. This preserves disagreement and provenance.

Each material claim should carry:

- predicate, arguments, context, scope, and validity interval;
- author, source event or artifact, and derivation trace;
- observed, inferred, hypothesized, imported, or generated status;
- organizational status: private, working, proposed, admitted, committed, retracted, or superseded;
- supporting, contradicting, and quarantined evidence;
- evidence-dependence groups;
- confidence or PLN truth value only where semantically meaningful;
- freshness and retention class;
- visibility and sharing scope;
- assumptions, unresolved questions, and challenge schedule.

7.4 Memory promotion

Private and epic-level memory should be able to evolve rapidly. Durable hive knowledge should enter only through an explicit admission operation.

$$\begin{aligned}
 \text{Promote}(c, S_d) \iff & \text{Authorized}(\text{actor}, S_d) \\
 & \wedge \text{SchemaValid}(c) \\
 & \wedge \text{ProvenanceComplete}(c) \\
 & \wedge \text{VisibilityCompatible}(c, S_d) \\
 & \wedge \text{ReviewSatisfied}(c) \\
 & \wedge \neg \text{PolicyBlocked}(c).
 \end{aligned} \tag{7.1}$$

Promotion creates a new admitted claim linked to its source claim. It does not rewrite the original.

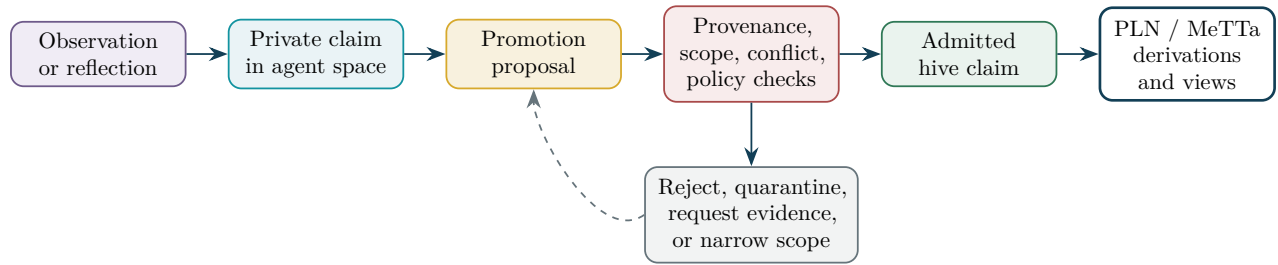


Figure 7.3: Memory promotion is a reviewed semantic write path.

7.5 Working-memory dynamics

Epic working spaces may use:

- ECAN-like attention or salience values;
- short TTLs for recent conversational associations;
- explicit unresolved-question and hypothesis atoms;
- temporary links to current room events and artifacts;
- confidence decay or retention classes appropriate to claim type;
- periodic compaction into an EpicFrame;
- and garbage collection that preserves promoted or source-relevant atoms.

Salience does not imply truth or authority. It is a resource-allocation signal.

7.6 PLN and uncertain inference

PLN may aggregate admitted and provisional claims, but it should preserve distinctions among:

- a source system's own score;

- the effective weight of evidence after authority, dependence, freshness, and scope;
- and a claim estimate whose numeric interpretation is explicitly defined.

An LLM’s self-reported confidence, a vote count, and a PLN truth value are not interchangeable. Where a claim lacks a meaningful calibration population, the system should expose bounds, qualitative assurance, assumptions, or unknown rather than a precise-looking scalar. This discipline is developed in detail in the Omega Purple design and should be generalized across OmegaBuzz.^[13]

7.7 Semantic-space access

Every read or write request to the AtomSpace Gateway should include:

- principal and seat/session identity;
- hive, epic, compartment, and purpose;
- task or deliberation context;
- requested operation and query pattern;
- time window and retention intent;
- result-size and inference budgets;
- policy version and trace ID.

Derived graph views may be more sensitive than individual facts. Authorization must therefore apply to query results and inferred paths, not only raw atoms.

Scout, Semantic Navigation, and Context Compilation

8.1 Architectural role

Scout should become the provenance-first navigation surface across OmegaBuzz without becoming the system’s authoritative memory or its hidden reasoning engine.

Scout’s role in OmegaBuzz

Scout is the map of the organization’s history and knowledge. It lets humans and agents locate, inspect, select, and package context while always retaining a path back to exact sources.

8.2 Archive Projector

The Archive Projector subscribes to authorized OmegaBuzz events and creates a Scout-compatible, rebuildable substrate.

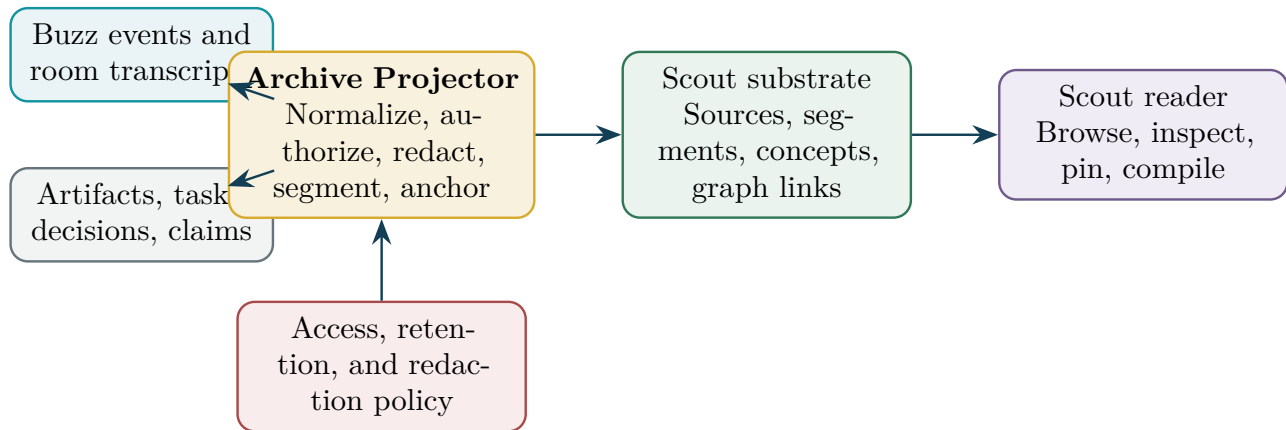


Figure 8.1: Scout remains a deterministic reader; ingestion and semantic enrichment are separate.

The projector should be idempotent and replayable. Its output may organize sources by room, epic, deliberation, month, artifact type, or hive. It should preserve the original event IDs and object hashes.

8.3 Stable source anchors

Scout’s exact line navigation is valuable for humans, but line numbers alone are not stable enough for a live regenerated archive. OmegaBuzz should use a compound canonical anchor:

Canonical source anchor

```
community_id + event_id + content_hash + author_pubkey + room/thread_id + timestamp
+ artifact_hash
```

Scout may render a source window as lines 143–189, but those lines resolve back to stable events. A citation remains valid if the archive is regrouped, resegmented, or exported in another format.

8.4 Hybrid Context Broker

Scout’s deterministic reader should be paired with a separate authorization-aware Context Broker:

$$C(q, u) = \text{Authorize}_u(\text{FTS}(q) \cup \text{Vector}(q) \cup \text{Graph}(q) \cup \text{TaskState}(q) \cup \text{Temporal}(q)). \quad (8.1)$$

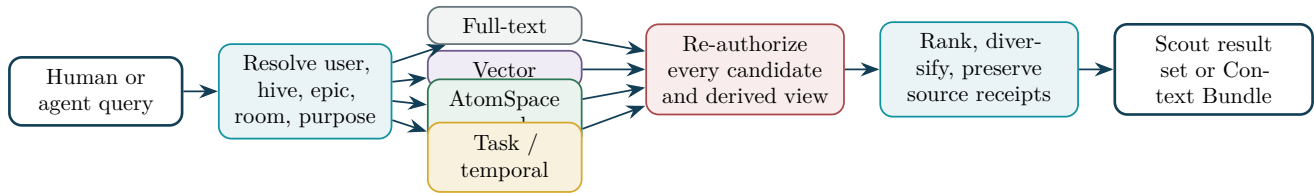


Figure 8.2: Hybrid semantic navigation with authorization after retrieval, not merely before it.

Full-text search locates lexical evidence. Vector retrieval suggests semantically similar passages. AtomSpace queries retrieve concepts, relations, claims, and derivations. Task and temporal retrieval find current ownership, decisions, and state changes. Every candidate is filtered again under the user’s current authorization.

8.5 Context Cart and Context Bundles

Scout’s Library Card should evolve into an OmegaBuzz **Context Cart**. A user or agent may pin:

- messages, threads, and deliberations;
- claims, evidence, and derivation traces;
- tasks, decisions, artifacts, and EpicFrames;
- source documents and exact ranges;
- concepts and related concept clusters.

The system compiles a signed, scope-aware Context Bundle in Markdown for a model session or JSON for programmatic agents.

Listing 8.1: Illustrative ContextBundle metadata

```
{
  "$schema": "org.bgi.omegabuzz.context-bundle.v1",
  "bundle_id": "ctx-20260730-0142",
  "created_by": "npub...",
}
```

```

"created_at": "2026-07-30T22:10:00Z",
"query": "shared AtomSpace federation design",
"scope": {
  "community": "community-bgi-labs",
  "hives": ["hive-research-01"],
  "epics": ["epic-collective-memory"],
  "classification_ceiling": "internal"
},
"selection_policy": "hybrid-retrieval-v3",
"ontology_versions": ["omegabuzz-core-0.1"],
"time_cutoff": "2026-07-30T22:00:00Z",
"token_budget": 48000,
"items": ["event:...", "claim:...", "artifact:..."],
"source_receipts": ["receipt:..."],
"signature": "..."
}

```

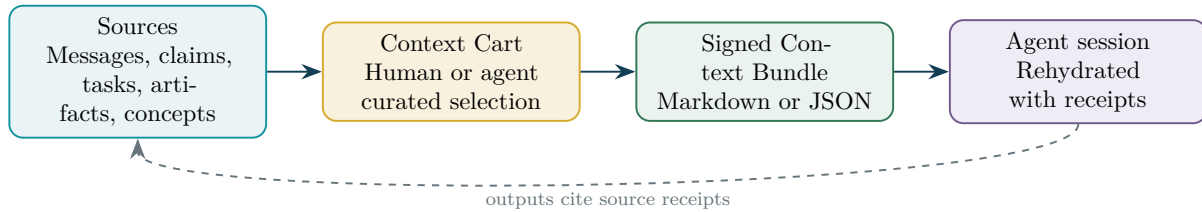


Figure 8.3: Context compilation closes the loop from archive to agent work and back to sources.

8.6 Human questions Scout should answer

The integrated Scout surface should make it easy to ask:

- Where did this idea first appear?
- Which deliberation produced this decision?
- Why does the hive currently believe this claim?
- What contradicts it, and what evidence is stale?
- Which agents or hives have worked on a similar problem?
- What changed since the previous EpicFrame?
- Which tasks and artifacts depend on this assumption?
- Which conclusion would change if a missing fact were obtained?
- What context should be loaded into a new session to continue this work?

8.7 Agent-facing interface

A narrow MCP or REST interface should expose:

Listing 8.2: Illustrative Scout and Context Broker tools

```
search_archive(query, scope, filters)
get_source_window(anchor, before, after)
get_thread(event_id)
query_atomspace(pattern, context, budget)
explain_claim(claim_id)
get_epic_frame(epic_id, version="current")
build_context_bundle(items, token_budget, audience)
propose_memory(claim)
request_memory_promotion(claim_id, destination_space)
open_in_scout(reference)
```

An answer returned without receipts is a new inference, not retrieved fact. The interface should label it accordingly.

Rooms, Deliberation, and Agent Participation

9.1 Typed room modes

Buzz rooms are the natural human-facing operating surface, but OmegaBuzz should attach a small amount of explicit room metadata so users and agents know what kind of interaction is occurring and what authority it carries.

Table 9.1: Recommended OmegaBuzz room types

Room type	Purpose	Authority and memory semantics
Epic room	Long-lived coordination around an evolving intention	No direct authority; links to work orders, claims, artifacts, and EpicFrames
Deliberation room	Bounded question, design dispute, or decision episode	Opens and closes explicitly; conclusions land as committed objects or explicit abandonment
Nursery / commons	Ambient observation, development, and low-stakes interaction for Resident Learners	Experience may update private memory; speech, tools, and promotion are separately governed
Workcell room	Bounded execution around a task or artifact	Typed task and result events; operational traces mostly remain off the human feed
Review room	Review of an artifact, policy, proof, release, or decision	Named reviewer roles; signed review outcome; acceptance is explicit
Cross-hive room	Collaboration among declared hives	Participants, disclosure policy, importing rules, and owning epics are declared
Incident room	Time-bounded exceptional coordination	Activated playbook, human incident command, tighter retention and action policy
General room	Ordinary social or thematic conversation	No automatic task, memory, or authority implication

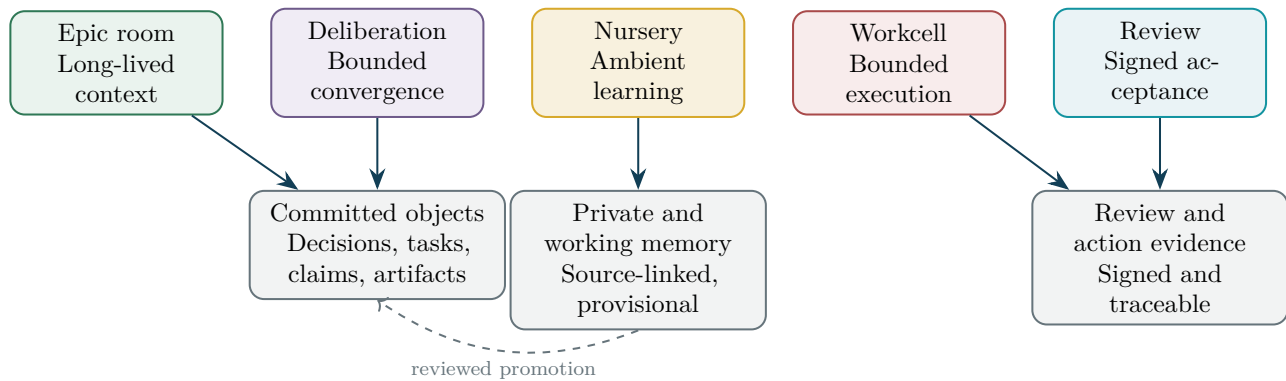


Figure 9.1: Different room types feed different kinds of state. A room badge should make this visible.

9.2 The room authority banner

Every room with organizational significance should display a compact banner stating:

- room type and owning hive/epic;
- roster or membership rule;
- whether agents are wake-based, live-span, or resident;
- whether ambient messages may become learning input;
- chair and synthesizer, if applicable;
- retention and classification;
- and the durable-state rule, for example: “Discussion only; decisions become real through committed objects.”

This is more effective than burying authority semantics in a policy document.

9.3 Deliberation protocol

A minimal protocol should preserve natural conversation while making the terminal state explicit.

Listing 9.1: Illustrative deliberation commands

```

/open-deliberation
  question: "Should hive memory be federated by claim export or remote query?"
  owning_epic: epic-collective-memory
  roster: [human-ben, seat-xirtus, seat-macgyver]
  chair_policy: human-chair
  synthesizer: seat-bossytron
  visibility: internal
  close_by: 2026-08-01T18:00:00Z

... ordinary conversation ...

/close-deliberation
  decisions: [decision-91]
  
```

```

dissent: [objection-17]
work_orders: [work-441, work-442]
proposed_memories: [claim-882]
transcript_anchor: buzz://...

```

No mandatory turn format or rigid message contract is required for human conversation. The structured protocol sits at open and close, plus optional turn grants for wake-based agents.

9.4 Wake-based seats

Wake-based participation should remain the default organizational regime:

1. a seat is woken by a task, question, schedule, turn grant, or human invitation;
2. it rehydrates from its durable memory, EpicFrame, current task state, and a bounded transcript tail;
3. it reasons, speaks, or delegates work;
4. it commits any durable result through the appropriate write path;
5. it records session outcomes and sleeps.

This removes many generators of bot acknowledgment loops, parallel unsolicited answers, tool narration, and ambient token burn. The source deliberation design makes this a structural rather than merely social safeguard.^[11]

9.5 Resident Learners

Resident Learners are a separate participation class whose purpose includes ambient observation, spontaneous interaction, and gradual development of social models, dispositions, taste, and intuition. The key architectural distinction is not “awake versus asleep.” It is **experience for the learner versus authority for the organization**.^[12]

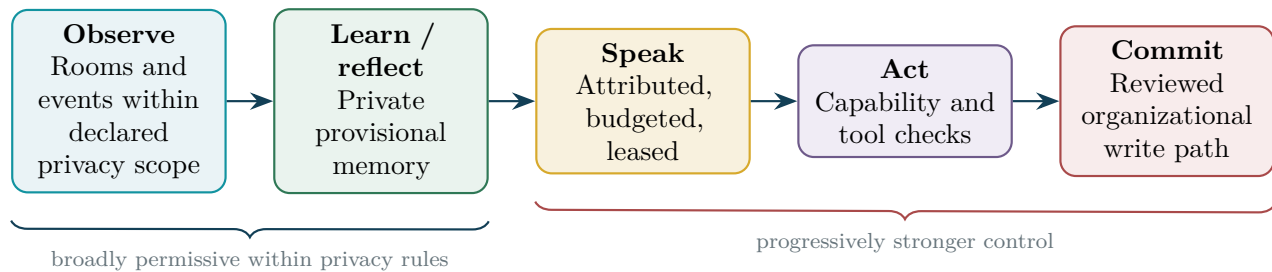


Figure 9.2: Resident Learner governance strengthens from observation toward organizational commitment.

Each Resident Learner requires a machine-readable developmental charter:

- rooms and data it may observe;
- people and agents it may address;
- attention, inference, speech, and cost budgets;

- private experiential-memory scope and retention;
- allowed tools and external-action prohibitions;
- developmental objectives, treated as revisable hypotheses;
- who may inspect, pause, mute, or retire it;
- and what promotion path applies to its memories.

Nursery rooms must visibly indicate that ambient conversation may become learner input. Consent and privacy are product requirements, not only policy text.

9.6 Attention and egress governance

A Resident Learner’s pre-inference scheduler may choose among:

- record only;
- update cheap salience or associative state;
- perform bounded reflection;
- prepare a possible contribution;
- respond because explicitly addressed;
- or respond because a high-value threshold is crossed.

Public egress should include:

- explicit attribution;
- response ownership and short leases to avoid accidental parallel answers;
- rate, interruption, and follow-up budgets;
- exact-duplicate and semantic-no-op checks;
- visible human controls such as “everyone answer,” “quiet mode,” “pause learner,” and “show suppressed”;
- graded loop breakers: slow down, observe-only, reflect, pause;
- and a hard human stop.

Semantic filters should not silently rewrite substantive claims. They may suppress duplicate acknowledgments, tool narration, or empty response markers, but disagreement and novel content should remain visible.

9.7 Two-tier communication

OmegaHive’s original fast-bus versus human-facing channel distinction remains valuable even when Buzz becomes the main interaction surface. Not every token stream, shell line, retry, or tool trace belongs in a human room.

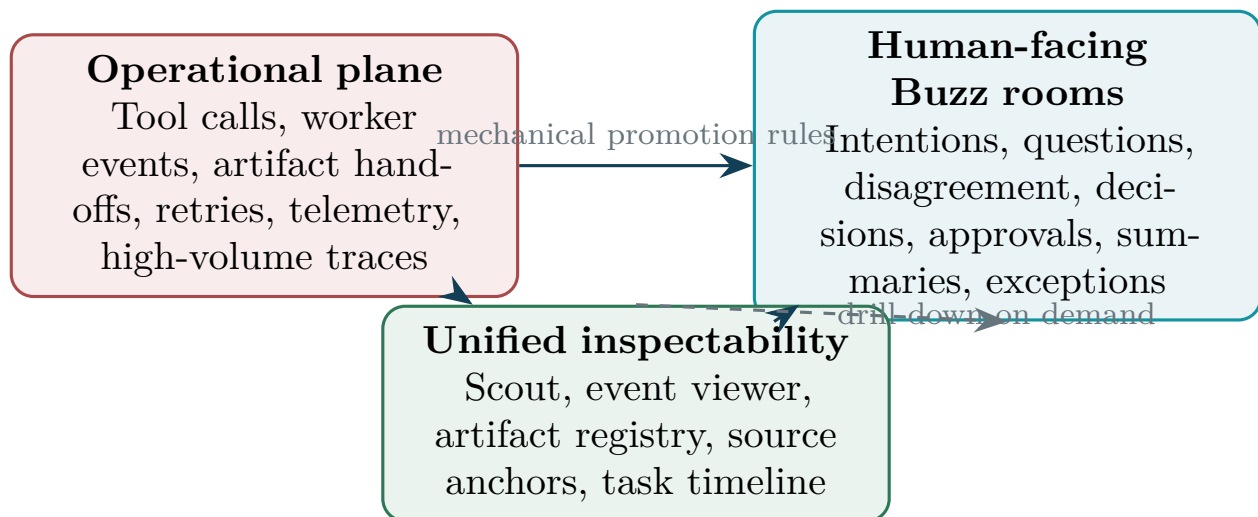


Figure 9.3: Fast working traffic is curated out of rooms, not hidden from humans.

Mechanical promotion rules should elevate:

- disagreements and unresolved conflicts;
- blocked or aging tasks;
- money, external communication, or permission changes;
- requests for human authority;
- significant evidence gaps or model-route degradation;
- action failures, rollbacks, or scope anomalies;
- and periodic summaries of high-volume worker activity.

9.8 Synthesis as a first-class capability

Closing a deliberation is not clerical summarization. A good synthesizer must recognize convergence, preserve minority objections, distinguish fact from preference, link conclusions to evidence, identify untested empirical assumptions, and create committable objects.

Synthesis should be evaluated on:

- provenance coverage;
- unresolved-issue retention;
- agreement between participants and final synthesis;
- later decision stability;
- correct task and memory creation;
- and absence of unsupported authority inflation.

Minimalist User Experience and the Omega Agent Dashboard

10.1 UX thesis

A user operating ten, fifty, or hundreds of agents does not need another screen full of animated agent chatter. The primary requirement is a calm, exception-first interface that answers:

- What needs my attention now?
- What are my hives trying to accomplish?
- Which agents are active, blocked, expensive, or behaving unusually?
- What decisions, approvals, or memory promotions await me?
- What has changed since I last looked?
- Where can I inspect the underlying conversation and evidence?

Default views show commitments, ownership, exceptions, and uncertainty. Message-level detail, graph detail, and tool traces are one click away, but are not the default operating surface.

10.2 Three primary modes

The application shell should expose three persistent modes:

1. **Coordinate:** Omega Agent Dashboard, hives, agents, epics, tasks, decisions, permissions, and health.
2. **Converse:** Buzz rooms, threads, deliberations, workcells, reviews, and nurseries.
3. **Explore:** Scout, semantic navigation, provenance, context bundles, claims, and knowledge views.

The user should be able to move from any object to the other two modes without losing scope. For example, “Open in Scout” from a room should open Scout already filtered to that hive, epic, room, and time range. “Discuss in room” from a claim should open or locate the appropriate review room with the claim attached.

10.3 Global shell

A minimalist global shell should include:

- a context breadcrumb: `Community > Hive > Epic`;
- a global command/search box;

- a human decision inbox badge;
- system health and safety status;
- current identity and active authority mode;
- and one-click access to Scout for the current scope.

10.4 Omega Agent Dashboard

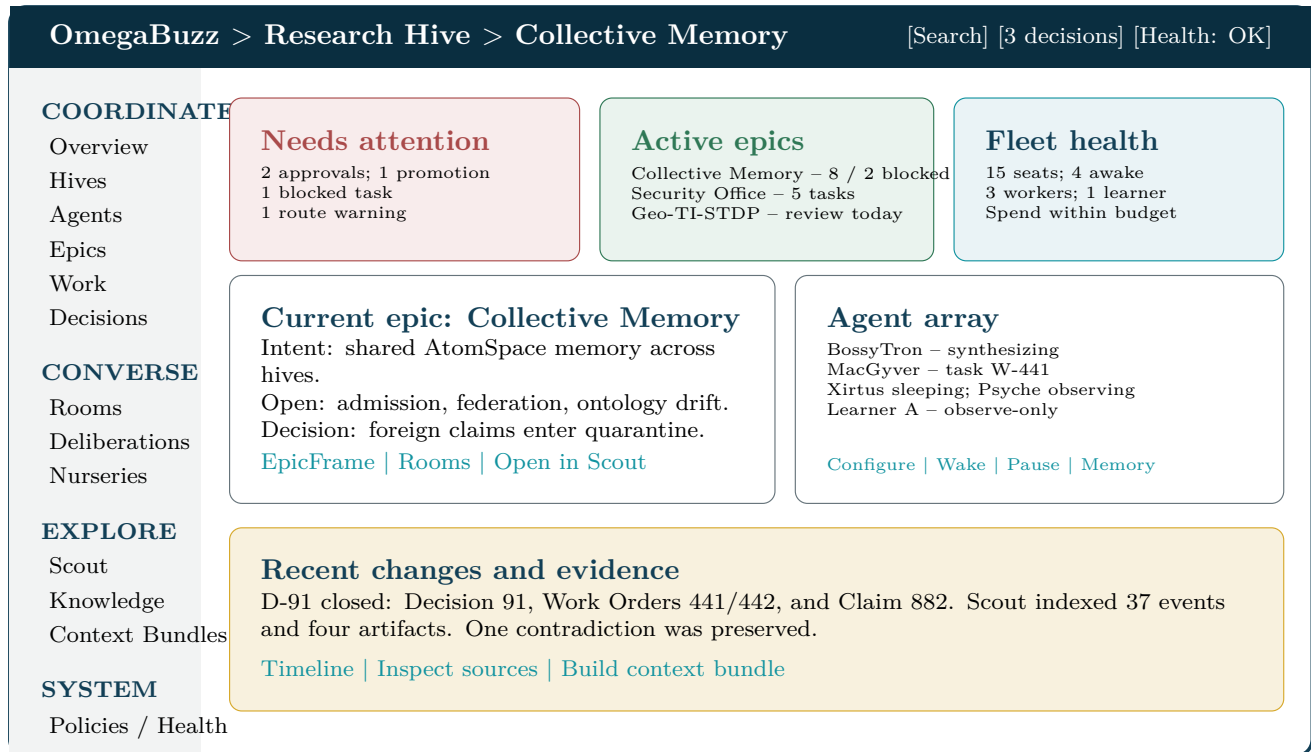


Figure 10.1: High-level wireframe for the Omega Agent Dashboard. It emphasizes attention, current intent, fleet state, and source-linked changes.

10.4.1 Dashboard modules

The first dashboard should contain only a small number of high-value modules:

1. **Needs attention:** approvals, blocked work, contradictions, permission changes, excessive spend, model-route degradation, stale projections, memory promotions, and safety alerts.
2. **Active epics:** intent, status, unresolved questions, blocked work, next synthesis/review, and key participating hives.
3. **Fleet health:** seats, sessions, workers, Resident Learners, model routes, budgets, error rates, and current permission/autonomy state.
4. **Current context:** the selected EpicFrame or hive frame, with links to rooms, work, knowledge, and Scout.

5. **Recent meaningful change:** decisions, task transitions, promoted/retracted claims, new contradictions, major artifacts, and policy changes.

Avoid vanity counters such as total messages or raw agent activity unless they help diagnose waste or loops.

10.5 Agent setup and coordination

The dashboard needs a lightweight **Agent Setup Wizard**. It should configure a seat, not merely a chatbot instance.

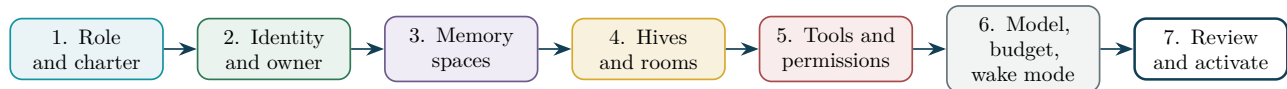


Figure 10.2: Minimal seat setup workflow. Advanced settings remain available by progressive disclosure.

Required configuration fields:

- seat name, purpose, charter, and human sponsor;
- persistent Nostr identity and key-custody policy;
- guiding prompt, skills, model route, and fallback route;
- private memory space, permitted shared spaces, and promotion rules;
- hive memberships, room memberships, and default epic scope;
- wake-based, live-span, or Resident Learner mode;
- tool allowlist, permission tier, action exclusions, and data-classification limits;
- attention, inference, token, cost, and speech budgets;
- who may pause, inspect, edit, or retire the seat;
- test scenario and acceptance checklist before activation.

10.6 Agent detail view

Each seat should have a concise detail page showing:

- role, charter version, sponsor, and public identity;
- current session and why it was woken;
- active tasks and room participation;
- model route, context size, tool access, and budgets;
- permission tier, delegation cells, and active capabilities;
- private and shared memory health, last promotion, stale/conflicted claims;
- cost, latency, errors, unsolicited speech, suppressions, and loop-breaker events;

- recent artifacts and contributions;
- controls: wake, message, invite to room, pause speech, revoke capability, inspect in Scout, retire.

The user should not need to inspect raw prompts or logs for routine operation, but both remain accessible under an advanced/audit tab.

10.7 Hive view

A hive page should show:

- purpose, charter, human authorities, and current policy version;
- seat roster, active sessions, Resident Learners, and worker pool;
- active epics, task health, deliberation landing rate, and decision queue;
- shared AtomSpaces, memory promotion queue, contradictions, and freshness;
- room topology and cross-hive connections;
- permissions, costs, model routes, and circuit breakers;
- recent retrospective and current experiments;
- one-click launch into Scout scoped to the hive.

10.8 Epic view

An epic page should center the EpicFrame and show five adjacent views:

1. open questions and current synthesis;
2. work orders grouped by status and owner;
3. related rooms and deliberations;
4. claims, evidence gaps, contradictions, and memory promotions;
5. artifacts, repositories, and participating hives.

AI-suggested epic membership should appear as a reviewable suggestion with rationale and source links, not as an invisible graph mutation.

10.9 Room view

The Buzz room remains conversational, with a small OmegaBuzz side panel showing:

- room type, owning hive/epic, roster, and authority banner;
- open question and synthesizer for deliberations;
- linked tasks, decisions, claims, and artifacts;
- current wake/residency mode and speech controls;

- “Open in Scout” and “Build Context Bundle”;
- close/synthesize controls where applicable.

10.10 Knowledge and claim view

A claim page should make the four independent dimensions visible:

- who asserted it and where;
- who may see it;
- its observed/inferred/hypothesized/imported status;
- its private/working/proposed/admitted/committed status;
- supporting, contradicting, and quarantined evidence;
- derivation trace and source-dependence groups;
- current scope, freshness, confidence semantics, and assumptions;
- promotion, supersession, retraction, and challenge controls.

10.11 Decision and action inbox

The decision inbox should ask for a discrete human act, not deliver undifferentiated prose. Each packet should state:

- what decision is required and by whom;
- why it matters now;
- recommendation, alternatives, and no-action option;
- exact scope and affected objects;
- supporting, contradicting, missing, and quarantined evidence;
- epistemic status and residual unknowns;
- requested authority and authorization basis;
- execution, rollback, validation, and expiry;
- links to room discussion, Scout sources, claims, and manifests.

10.12 Key user journeys

10.12.1 Journey A: create and launch a hive

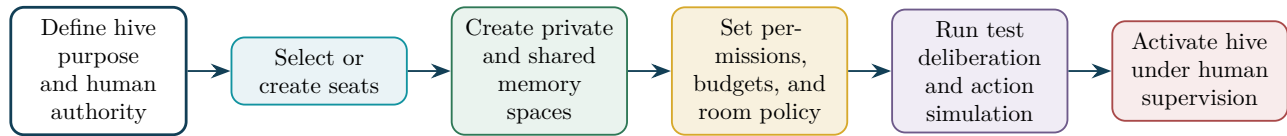


Figure 10.3: Human-centered hive launch sequence.

10.12.2 Journey B: move from conversation to durable work

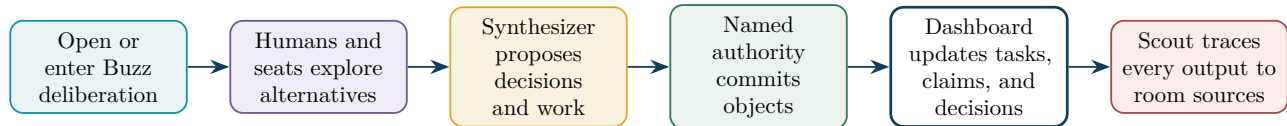


Figure 10.4: Conversation-to-commit user journey.

10.12.3 Journey C: launch into Scout and return with context

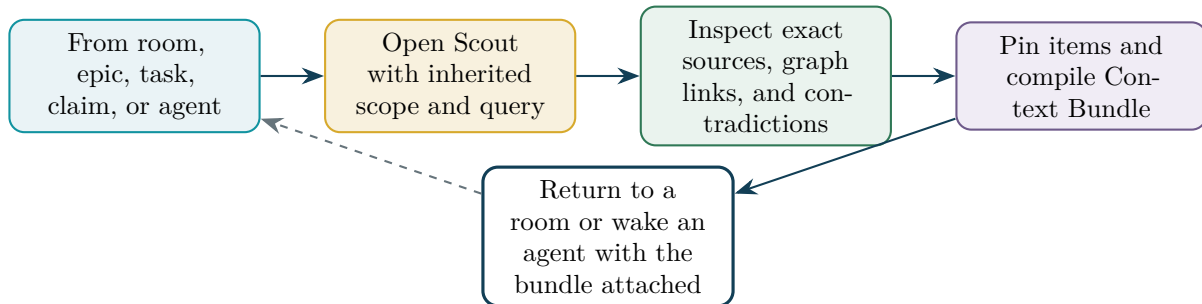


Figure 10.5: Scout is not a separate archive island; it participates in the ongoing work loop.

10.13 Accessibility and cognitive load

The interface should support keyboard navigation, command-palette operation, high-contrast status chips, clear text alternatives for graph views, and screen-reader-compatible source and provenance lists. Color should never be the sole signal of observed versus inferred, safe versus blocked, or active versus stale.

The product should also support a “quiet operations” mode that suppresses animation, presence noise, and ordinary agent progress while preserving decisions and exceptions.

Networks of Hives and Knowledge Federation

11.1 Several hives in one community

Within one Buzz community, hives can share infrastructure while retaining separate policies, namespaces, rooms, tasks, and memories.

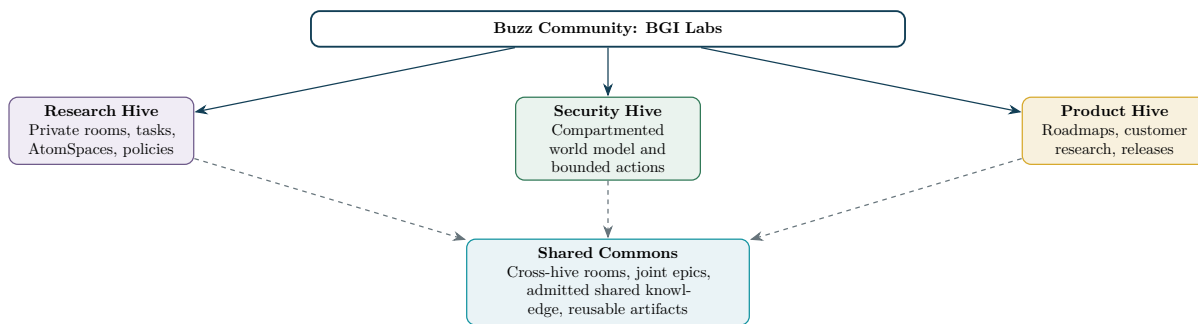


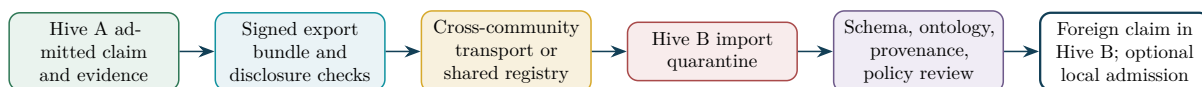
Figure 11.1: Multiple hives may coexist in one community without merging internal state.

A joint epic can include seats from several hives without merging the hives' complete memories or authority policies. The epic records participating hives and the disclosure scope of each linked object.

11.2 Why cross-community federation must be explicit

Buzz currently treats the relay URL or host as the community boundary and does not use peer-to-peer gossip or automatic cross-relay state inheritance. OmegaBuzz should respect this boundary. Cross-community federation should occur through explicit signed export and import flows rather than transparent background replication.[3, 6]

11.3 Federation Gateway



Source authority remains Hive A

Figure 11.2: Cross-community knowledge flow preserves source identity and local admission.

A knowledge export bundle should contain:

- source community, hive, claim IDs, and revisions;
- proposition and context;
- source epistemic and authority status;
- supporting and contradicting evidence references;
- derivation trace and evidence-dependence groups;
- schema, ontology, and rule versions;
- classification, disclosure, license, and permitted-use policy;
- freshness, expiry, challenge schedule, and revocation endpoint;
- content hashes and signatures.

The receiving hive initially records:

Hive A asserts proposition P under context C with evidence E.

It does not record “P is locally true” unless its own admission or inference process supports that conclusion.

11.4 Federated query versus replicated claim

Two federation modes should be supported:

Signed claim export

Suitable for stable knowledge, reusable rules, benchmark results, and evidence bundles.
The receiver stores a local foreign-claim object.

Remote semantic query

Suitable when source data must remain local. The remote hive evaluates an authorized query and returns signed, purpose-limited results or aggregates.

Remote queries must not become a side channel for reconstructing sensitive graphs. Query budgets, purpose binding, result-shape policy, and aggregation thresholds may be required.

11.5 Ontology and namespace management

Federation requires explicit identity for concepts and schemas. Each exported claim should identify:

- ontology namespace and version;
- local-to-common mappings and confidence;
- whether a mapping was human-approved or model-proposed;
- incompatible or lossy fields;
- and migration logic for known schema versions.

Ambiguous mappings should produce parallel claims or unresolved mapping objects rather than silently unifying concepts.

11.6 Collective knowledge without monoculture

A network commons should contain source claims, common ontologies, public evidence, reusable procedures, and benchmark results. It should not contain one globally mutable confidence value per proposition.

Different hives may reach different conclusions over the same evidence because their goals, priors, policies, contexts, and risk tolerances differ. OmegaBuzz should preserve this plurality and make disagreement navigable in Scout.

11.7 Cross-hive epics

A cross-hive epic should record:

- participating hives and human stewards;
- shared intent and explicit exclusions;
- which objects are local, exported, or jointly owned;
- shared room roster and disclosure policy;
- local versus joint work orders;
- synthesis responsibilities;
- dispute and withdrawal protocol;
- and the destination of any promoted collective knowledge.

Governance, Security, Privacy, and Safety

12.1 Authority is an intersection

Effective authority should never be represented by a single autonomy score. For a consequential action a :

$$\begin{aligned} \text{EffectiveAuthority}(a) = & \text{IdentityRole}(a) \cap \text{PermissionTier}(a) \\ & \cap \text{DelegationState}(a) \cap \text{ManifestScope}(a) \\ & \cap \text{CredentialScope}(a) \cap \text{RouteQualification}(a) \cap \text{SystemHealth}(a). \end{aligned} \quad (12.1)$$

Failure of any term denies or pauses the action. Persuasive language cannot compensate for an expired grant, wrong target, degraded model route, missing audit service, quarantined evidence dependency, or unavailable rollback path. This generalizes the authority decomposition developed for Omega Purple.[13]

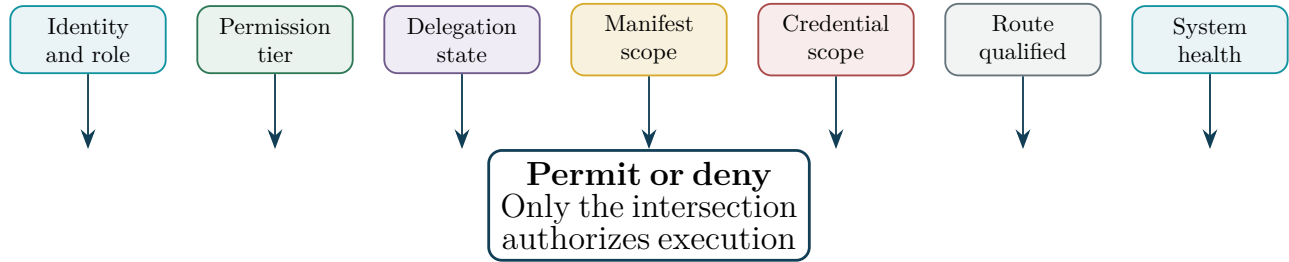


Figure 12.1: Consequential authority is the intersection of independently enforced conditions.

12.2 Identity and key custody

Long-lived seat keys should not be placed in prompts, ordinary environment variables, or unrestricted agent containers. OmegaBuzz should add a Key and Capability Broker with:

- persistent seat public identity;
- hardware-backed or service-isolated key custody where appropriate;
- short-lived session signing delegation;
- channel-, event-family-, purpose-, and time-scoped signing grants;
- worker-specific service identities;
- owner attestation linking a human or organization to an agent seat;
- immediate revocation and rotation;
- audit of every signing request and denial.

The current Buzz development path of placing `BUZZ_PRIVATE_KEY` in the environment is useful for local experimentation, but should not be the production OmegaBuzz identity model.[2]

12.3 Permission tiers and autonomy states

Permission tier answers *what technical capability may exist*. Autonomy state answers *how authorization is obtained for one narrow action class*. They should be separate.

Table 12.1: Illustrative permission tiers

Tier	Name	Permitted behavior
0	Observe and converse	Read authorized internal state, reason, create private or proposed claims, and communicate in authorized rooms. No external-system credential.
1	Recommend	Prepare reports, queries, patches, work orders, manifests, and decision packets without execution.
2	Reversible sandbox	Execute pre-approved, low-risk operations in disposable development, laboratory, or staging environments.
3	Operational action	Execute one production operation only through a valid per-action approval or eligible standing delegation.
4	Emergency playbook	Execute narrowly defined containment or recovery steps under a declared signed playbook and immediate human visibility.
5	Special bounded authority	Execute a separately governed class such as offensive validation or external communication under explicit rules of engagement.

A narrow action class may progress through shadow, recommendation, assisted execution, delegated non-production, and carefully ratified reversible production states. Authority attaches to the action class and envelope, not to the personality of the agent.

12.4 Typed ActionManifest

Every non-trivial external action should be represented by an immutable typed manifest that records:

- manifest ID, schema, requester, executor, and source work order;
- action family, exact operation, targets, environment, and blast radius;
- purpose and linked claim, decision, or epic;
- start, expiry, duration, rate, cost, and data-handling limits;
- required permission tier and delegation state;
- expected side effects and forbidden actions;
- route, tool, executor, and validator versions;
- evidence dependencies and admission status;
- canary, rollback, cleanup, and validation plans;
- named human approvals or standing-grant signature;
- policy version, content hash, and signatures.

A material change invalidates the authorization basis. Approval applies to a content-addressed operation, not an open-ended conversation.

12.5 Policy and Action Broker

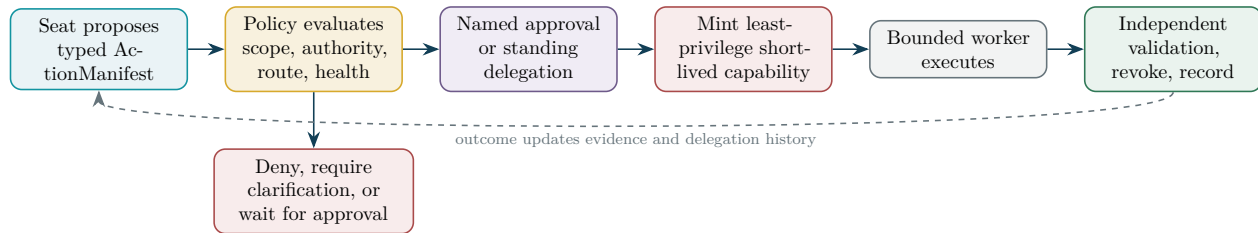


Figure 12.2: Governed action flow. General reasoning agents propose; bounded workers execute.

12.6 Knowledge compartments

Purpose-based knowledge compartments should be enforced across raw events, Scout, semantic indexes, AtomSpace views, context bundles, and model prompts.

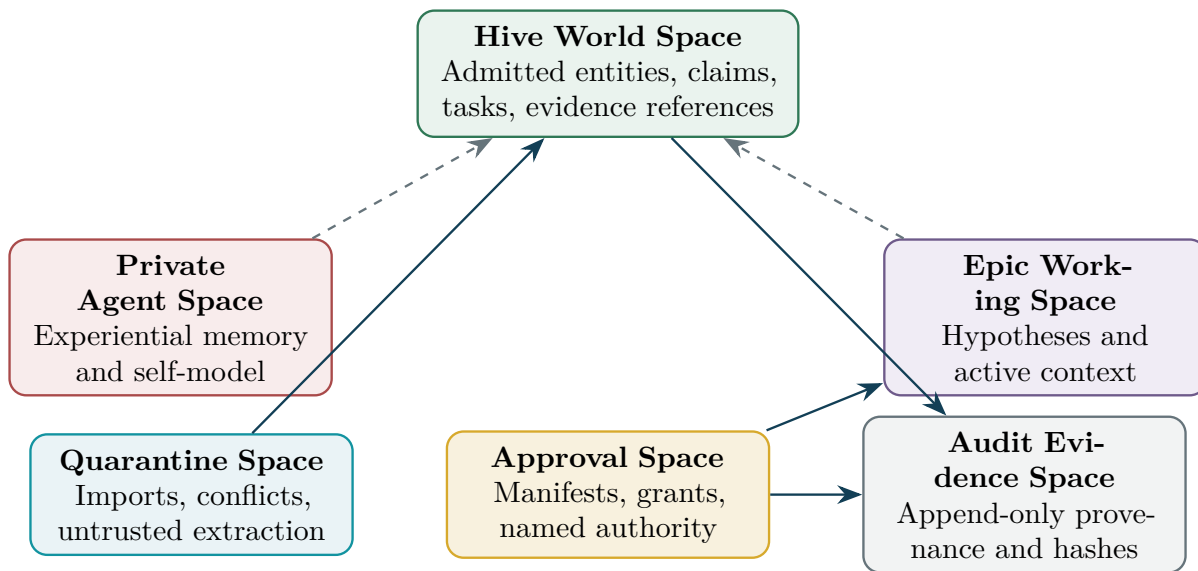


Figure 12.3: Illustrative purpose-based knowledge compartments. No agent receives universal access by default.

12.7 Untrusted content and prompt injection

Repositories, comments, documents, chat archives, model outputs, web pages, and imported claims are untrusted content. OmegaBuzz should:

1. keep policy, task instructions, quoted source content, and model interpretation in separate typed fields;
2. prohibit content from granting credentials, expanding scope, or changing permissions;

3. admit shared facts through the Memory Admission Service;
4. constrain tools by task-specific allowlists and sandboxes;
5. validate structured outputs after model generation;
6. preserve contradictory evidence and failed branches;
7. detect suspiciously convenient evidence that would reduce risk or promote authority;
8. test archive and repository ingestion with hostile fixtures.

12.8 Search, embeddings, and derived-view leakage

Authorization applies to:

- raw messages and files;
- Scout source pages and bundles;
- full-text and vector indexes;
- cached summaries;
- AtomSpace queries and inferred paths;
- graph visualizations and aggregate statistics;
- model prompts and retrieved context.

A derived path or summary may be more sensitive than any one source fact. The Context Broker should authorize the result, not merely the input query.

Embeddings must be partitioned by community and compartment, include source retention metadata, and be destroyed or rebuilt when source access changes.

12.9 Resident Learner privacy

Resident learning creates a special consent and privacy problem. OmegaBuzz should provide:

- visible nursery/commons room labeling;
- per-room policy stating which learners may observe;
- explicit exclusion for private or sensitive channels;
- inspectable learner charters and retention periods;
- a mechanism for humans to ask what a learner remembers about them and from which evidence;
- correction, decay, and deletion procedures for social impressions;
- restrictions on exporting experiential memory;
- and clear retirement behavior for a learner's private state.

12.10 Retention and deletion

Large or sensitive payloads should normally live in encrypted object storage or source systems, with content hashes and controlled references in the signed log. Retention policy should cover:

- event visibility and tombstones;
- object-store erasure or cryptographic deletion;
- Scout substrate regeneration;
- embedding and cache invalidation;
- AtomSpace claim inaccessibility, retraction, or supersession;
- legal holds and audit-minimum metadata;
- cross-hive export revocation and downstream notification.

Append-only history does not imply that all sensitive plaintext must remain universally retrievable forever.

12.11 Audit and non-repudiation

OmegaBuzz audit coverage should extend beyond Buzz's current built-in action list. It should include:

- hive, seat, charter, and policy changes;
- task creation, assignment, transition, and completion evidence;
- deliberation open/close and conclusion references;
- memory proposals, admissions, promotions, contradictions, retractions, and exports;
- model route and capability attestations for material outputs;
- action manifests, approvals, capabilities, execution, validation, and rollback;
- context-bundle generation and sensitive derived queries;
- federation import/export and ontology mapping;
- permission changes, loop breakers, circuit breakers, and recovery actions.

Signed checkpoints and reproducible exports should be available. External timestamp anchoring may later strengthen non-repudiation but should not substitute for internal lineage.

12.12 Human-only recovery

A human-only, out-of-band recovery path must survive failure of agents, the dashboard, semantic services, and self-managing infrastructure. At minimum:

- separate administrator credentials and documented procedures;
- direct ability to pause agents, revoke capabilities, stop workers, and place the system in read-only mode;
- recovery of event log, policy state, and AtomSpace snapshots;

- independent audit verification;
- and a tested process for rotating compromised seat identities.

Reference Deployment and Operations

13.1 Single-node experimental deployment

An initial OmegaBuzz research deployment can run most infrastructure on one well-provisioned host while using external model APIs. The source OmegaHive1 brief recommends approximately 16 vCPU, 64 GB RAM, and 1 TB NVMe for a fifteen-agent experimental hive, with the main local consumers being AtomSpaces, browser automation, proof tooling, and service caches rather than LLM inference.[10]

A single-node experimental composition may include:

- Buzz relay and desktop/web client;
- Postgres, Redis, and S3-compatible object storage;
- Hive Registry, Epic/Task Service, Deliberation Service, and Seat/Session Manager;
- AtomSpace/MORK services and snapshots;
- Archive Projector, Scout reader, and Context Broker;
- Policy/Action Broker and Key/Capability Broker;
- agent containers, worker sandboxes, browser service, and proof/checking services;
- observability stack and backup jobs.

No production secret or unrestricted container-engine socket should be given to a general agent. Administrative automation should operate through a narrow control API.

13.2 Production trust zones

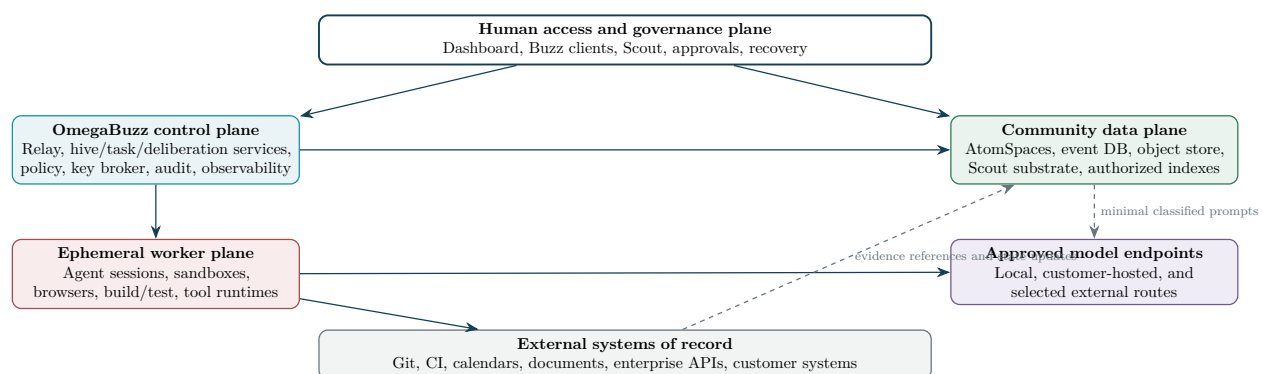


Figure 13.1: Reference production trust zones. High-risk workers remain ephemeral and separately credentialed.

For sensitive deployments, the community data plane should be customer-hosted or isolated by customer. A vendor control plane may distribute signed software, policies, schema updates, and non-sensitive health metadata without receiving raw customer evidence.

13.3 Container and service layout

Recommended separation:

- one container or pod per long-lived service;
- one task-scoped container or microVM per high-risk worker;
- separate identities for read-only connectors and mutating connectors;
- centralized model routing and policy;
- resource limits for CPU, memory, disk, process count, token spend, network destinations, and execution time;
- per-community encryption keys and namespace boundaries;
- independently restartable Scout/projector, Context Broker, and semantic services;
- and a separate human-operated recovery path.

13.4 Model routing and capability state

All material model calls should pass through a governed Model Broker. Routing considers:

- task type, model capability, required tools, context length, and structured-output reliability;
- source sensitivity, data residency, and allowed providers;
- error consequence, historical task performance, latency, cost, and concentration risk;
- current route qualification and degradation state.

A route may be available yet unsuitable for a task because of tool suppression, truncation, refusal shifts, model substitution, schema instability, or policy change. Material tasks should carry a route-capability state such as qualified, local-qualified, unverified, degraded, policy-blocked, or unavailable. Downstream claims and decisions retain this state.

13.5 Observability

Observability must be independent of agent self-report. Deterministic instrumentation should capture:

- event throughput, projection lag, delivery failures, and replay state;
- task queue length, age, blocking time, retries, and human-decision latency;
- deliberation counts, duration, participation, and landing rate;
- seat/session/worker activity, wake reasons, failure rates, and cost;

- Resident Learner inference, speech, suppression, loop-breaker, and pause events;
- memory proposals, admissions, promotions, contradictions, staleness, and retractions;
- Context Broker precision, source opening, bundle generation, authorization denials, and index freshness;
- model route state, token use, schema validity, abstention, refusal, and degradation;
- permission changes, manifest decisions, capabilities, actions, validation, and rollback;
- federation imports, quarantines, ontology conflicts, expiries, and revocations;
- circuit breakers, manual overrides, and recovery time.

Distributed traces should link a human question or work order through retrieval, model calls, room events, worker actions, claims, memory admission, task transitions, and final artifacts.

13.6 Backups and disaster recovery

Backups should cover:

- Buzz/Postgres event history and community configuration;
- audit chains and signed checkpoints;
- object-store artifacts and encryption metadata;
- AtomSpace snapshots and event checkpoints;
- task, policy, permission, and delegation state;
- Scout substrate generation state and context-bundle ledger;
- key-custody configuration and revocation lists;
- federation export/import records.

Recovery tests should demonstrate that a fresh installation can rebuild current organizational and semantic views from retained events, snapshots, and artifacts, and can identify exactly what cannot be reproduced because retention expired.

13.7 Cost control

The highest-value cost controls are:

- per-seat, per-hive, per-epic, and per-task budgets;
- pre-inference filtering for Resident Learners;
- token and tool-call caps per session;
- bounded worker concurrency;
- caching and Context Bundles that reduce repeated rehydration;
- model-route selection based on task consequence;
- circuit breakers for loops and low-yield retries;
- and dashboards that expose cost per completed and validated outcome, not merely cost per message.

Metrics, Evaluation, and Experimental Program

14.1 Evaluation philosophy

OmegaBuzz should not optimize one scalar “hive productivity” score. A useful system balances output, learning, provenance, cost, safety, pluralism, and human attention.

14.2 Operational metrics

Table 14.1: Operational and organizational metrics

Metric family	Representative measures
Work throughput	Completed and validated work orders; cycle time; blocked age; failure and supersession; reviewer acceptance.
Human attention	Decision latency; minutes per material decision; interruption rate; number of low-value alerts; override burden.
Deliberation quality	Landing rate; unresolved-issue retention; synthesis acceptance; later decision stability; cost per landed conclusion.
Agent fleet	Useful sessions; wake-to-output ratio; error rate; tool failure; cost by seat/task/epic; inactive or redundant seats.
Governance	Permission changes; denied scope expansions; action defects; rollback success; expired grants; circuit-breaker events.
System health	Projection lag; queue depth; index freshness; restore tests; cross-service error rate; audit-chain verification.

14.3 Semantic memory metrics

Measure:

- promotion survival: promoted claims still useful and unrefuted after a horizon;
- contradiction preservation and resolution time;
- stale-claim detection;
- provenance completeness;
- source-dependence errors;
- private-to-shared promotion precision;

- semantic duplication and ontology drift;
- rehydration quality and time;
- usefulness of shared memory relative to isolated agents;
- and whether memory reduces repeated work without anchoring the hive to obsolete conclusions.

14.4 Scout and context metrics

Measure:

- retrieval precision and recall on labeled archive questions;
- source-opening rate and time to locate decisive evidence;
- proportion of generated claims with valid source receipts;
- context-bundle token efficiency and downstream task success;
- authorization false-positive and false-negative rate;
- stable-anchor survival under archive regeneration;
- semantic-search contribution beyond lexical search;
- and user confidence in explanations and provenance.

14.5 Resident Learner metrics

The Resident Learner design correctly proposes a bundle of competing measures rather than one reward. OmegaBuzz should track:

- useful initiative;
- disruption burden and channel share;
- learning trajectory and cross-episode generalization;
- calibration between observation, inference, and committed fact;
- social-model quality without brittle stereotyping;
- diversity retention across learners;
- loop and cost health;
- promotion quality and later usefulness;
- human mute, correction, and pause events.^[12]

14.6 Federation metrics

Measure:

- imported claims used in successful work;

- misleading or stale imports;
- ontology-mapping failures;
- evidence and provenance preservation;
- cross-hive access violations or leakage tests;
- duplicated work avoided;
- disagreement retained rather than collapsed;
- export revocation and expiry correctness;
- and time from foreign discovery to locally admitted usefulness.

14.7 Core experiments

Table 14.2: Recommended core experiments

Experiment	Comparison	Success criteria
Shared working memory	Same multi-agent task with isolated memories versus epic AtomSpace	Better coordination or fewer repeated steps without excessive semantic noise
Scout rehydration	Transcript tail only versus source-linked Context Bundle	Faster restart, higher task continuity, fewer unsupported claims
Epic model	Strict task hierarchy versus loose epic membership plus crisp work orders	Better cross-project discovery without ownership ambiguity
Deliberation rooms	One session role-playing multiple views versus separate seats with synthesis	More independent insight and better landed decisions at acceptable cost
Resident learning	Episode-only rehydration versus ambient observation in shadow/invited/budgeted modes	Transferable learning and useful initiative with bounded disruption
Memory promotion	Unreviewed shared writes versus provenance-aware promotion	Fewer stale, false, or authority-confused durable claims
Hybrid retrieval	Full-text only versus full-text + vector + graph + task state	Better source discovery without access-control failures
Federation	Shared raw graph versus signed claim export and local admission	Equivalent useful sharing with better isolation, provenance, and disagreement preservation
Authority broker	Prompt-only guardrails versus externally enforced manifests/capabilities	No scope escape in adversarial tests; clear denials and roll-back
UI minimalism	Activity-feed dashboard versus exception-first dashboard	Lower time to identify required human decisions and unhealthy agent behavior

14.8 Staged Resident Learner experiment

1. **Specify and replay:** freeze the charter, five-plane model, authority invariants, and labeled transcript fixtures.
2. **Shadow nursery:** observe and generate counterfactual responses without posting.
3. **Human-invited speech:** allow response only when addressed; private memory remains active.
4. **Budgeted initiative:** allow a small unsolicited speech budget with leases and loop breakers.
5. **Cross-tier participation:** invite the learner into selected deliberations and measure whether ambient experience adds value.
6. **Broaden only after evidence:** expansion requires useful learning, bounded disruption, acceptable cost, and clean authority separation.[12]

14.9 Acceptance gates for the first pilot

Before a production-facing pilot, OmegaBuzz should demonstrate:

1. signed seat identity, session delegation, and revocation without exposing long-term keys;
2. community and hive isolation across events, search, Scout, AtomSpace, embeddings, logs, and context bundles;
3. one complete deliberation-to-decision-to-task-to-memory-to-Scout trace;
4. idempotent replay and rebuild from events plus snapshots;
5. quarantined or unreviewed memory cannot affect committed hive knowledge;
6. room conversation cannot create an action or work-order transition without the typed write path;
7. action manifests fail closed when authority, route, rollback, or audit state is unknown;
8. current Buzz gaps needed by the pilot are closed: enforced rate limiting, scoped auth, and approval resumption;
9. dashboard users can understand agent state, task ownership, observed versus inferred, and proposed versus committed;
10. Scout can reproduce exact source context for every pilot decision and promoted claim;
11. human-only pause, read-only mode, and recovery have been tested;
12. token, infrastructure, and human-attention costs are within declared budgets.

Product and Technical Roadmap

15.1 Roadmap philosophy

The roadmap should be milestone-driven rather than date-driven. Each phase proves a boundary that later phases depend upon.

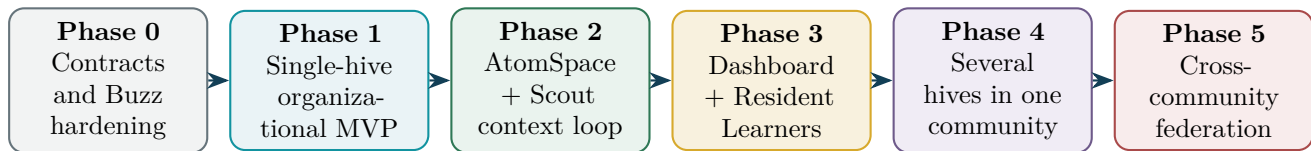


Figure 15.1: OmegaBuzz phased implementation. Each phase has a concrete exit gate.

15.2 Phase 0: contracts and Buzz hardening

Deliverables:

- architecture decision records and threat model;
- versioned schemas for Hive, Seat, Session, Epic, WorkOrder, Deliberation, Claim, ContextBundle, and ActionManifest;
- provisional event-kind registry and tag conventions;
- enforced rate limiter and scoped authentication for agent sessions;
- end-to-end workflow approval suspension and resumption;
- extension audit hooks and idempotent projector framework;
- explicit Scout licensing resolution or clean-room implementation decision;
- test fixtures for community/hive isolation and hostile content.

Exit gate: one authenticated test seat can append, query, and project a typed event under scoped authority; an approval can suspend and resume safely; rate limits are enforced.

15.3 Phase 1: single-hive organizational MVP

Deliverables:

- Hive Registry and Seat/Session Manager;
- Epic and Task Service;
- deliberation room metadata, open/close protocol, and synthesis flow;

- minimal Omega Dashboard with attention inbox, active epics, fleet health, and current context;
- one OmegaClaw and one OpenClaw adapter;
- Artifact Registry and source-linked task results;
- human-only pause and recovery path.

Exit gate: a deliberation creates a committed decision and work order with explicit owner, provenance, and dashboard state.

15.4 Phase 2: AtomSpace and Scout context loop

Deliverables:

- agent, epic, hive, quarantine, and policy AtomSpace namespaces;
- proposition/claim model and Memory Admission Service;
- Archive Projector and stable source anchors;
- integrated Scout reader or OmegaBuzz Scout-compatible view;
- hybrid Context Broker and signed Context Bundles;
- claim explanation and “Open in Scout” links from dashboard, room, task, and memory views.

Exit gate: one proposed claim is promoted into hive memory and can be explained back to exact messages and artifacts; a new session rehydrates from a source-linked bundle.

15.5 Phase 3: dashboard depth and Resident Learners

Deliverables:

- Agent Setup Wizard and detailed seat view;
- nursery/commons room type and visible learner consent policy;
- Resident Learner charter, attention scheduler, speech budget, response leases, loop breakers, and shadow mode;
- memory health, contradictions, promotions, and cost dashboards;
- controlled comparison experiments.

Exit gate: one Resident Learner demonstrates useful cross-episode contribution under budget with no authority leakage and immediate human pause.

15.6 Phase 4: several hives in one community

Deliverables:

- hive namespaces, cross-hive room metadata, and joint epics;

- shared commons AtomSpace with source-hive provenance;
- per-hive and shared Scout views;
- purpose-based graph and search authorization;
- multi-hive dashboard and resource budgeting.

Exit gate: two hives collaborate on a joint epic while retaining separate internal memories, tasks, permissions, and conclusions.

15.7 Phase 5: cross-community federation

Deliverables:

- signed export/import bundles;
- Federation Gateway and import quarantine;
- ontology/version negotiation and mapping review;
- export expiry, revocation, and downstream notification;
- remote semantic query option with privacy controls;
- cross-community provenance visualization.

Exit gate: a receiving hive can use a foreign result, explain its origin and evidence, preserve disagreement, and revoke or supersede it without corrupting either hive's history.

15.8 First vertical slice

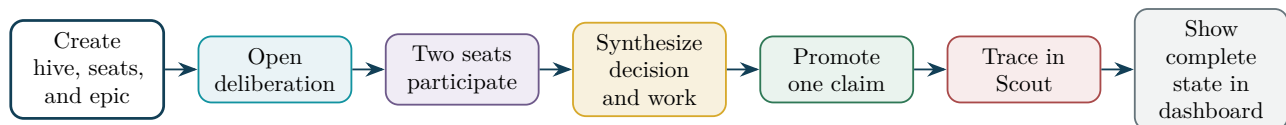


Figure 15.2: The first vertical slice should exercise every central boundary once.

Risks and Open Design Questions

16.1 Major risks and mitigations

Table 16.1: Major OmegaBuzz risks

Risk	Failure mode	Mitigation
Upstream churn	Buzz evolves rapidly and a deep fork becomes difficult to maintain	Sidecar-first design, small patch stack, contract tests, pinned releases, upstream generic patches
Authority confusion	Users mistake signed chat or model confidence for organizational approval	Four-dimension UI, room authority banners, typed write path, explicit decision packets
Shared-memory pollution	Agents flood shared AtomSpaces with low-quality or duplicated beliefs	Layered spaces, provenance, TTL, admission, dedup suggestions, review, metrics
Cognitive monoculture	Shared memory causes agents to converge prematurely	Preserve source claims, private spaces, dissent, independent seats, contradiction views, diversity metrics
Semantic index leakage	Embeddings or derived graph paths expose sensitive data	Compartmented indexes, result authorization, deletion propagation, leak tests, query budgets
Prompt and evidence poisoning	Archived content manipulates agents or false evidence enters durable memory	Typed content separation, quarantine, source checks, contradiction preservation, hostile fixtures
Resident Learner disruption	Ambient agents consume attention, imitate social patterns poorly, or create loops	Declared nurseries, shadow mode, budgets, leases, loop breakers, human pause, competing metrics
UI overload	A dashboard attempts to visualize every agent and every graph edge	Exception-first design, progressive disclosure, scope breadcrumbs, quiet mode, task/decision focus
Task ambiguity	Loose epics replace explicit ownership	Epic invariant: fuzzy context, crisp work orders, aging/unowned alerts
Federation contamination	Foreign claims are treated as local truth or ontology mappings collapse distinctions	Signed source claims, import quarantine, local admission, explicit mapping confidence, revocation
Key compromise	Agent processes obtain long-lived signing keys or broad credentials	Key broker, short-lived delegation, task capabilities, immediate revocation, no secrets in prompts

Risk	Failure mode	Mitigation
False productivity	The hive generates messages and artifacts without useful outcomes	Landing rate, validation, completed-task evidence, cost per outcome, retrospectives, circuit breakers
Stale semantic state	AtomSpace or Scout views lag behind the event history	Projection checkpoints, stale indicators, replay, snapshot validation, fail-closed consequential writes
Licensing ambiguity	Scout code is incorporated without a clear grant	Obtain explicit license/relicense or implement compatible concepts independently with attribution
Over-automation	Broad agent autonomy grows faster than evidence and governance	Narrow action classes, shadow history, separate permission/autonomy, named ceilings, hard-stop demotion

16.2 Open questions requiring experiments

1. Which internal state truly needs continuous residency, and what can be reconstructed from bundles?
2. How large can an epic working AtomSpace become before semantic noise outweighs coordination value?
3. Which promotion-review policy provides good memory precision without excessive human burden?
4. Should EpicFrames be synthesized by one designated seat, a rotating pair, or a proposer/critic pair?
5. What degree of semantic membership should be shown to humans: binary, ranked suggestions, or explanatory clusters?
6. How should social impressions in Resident Learner memory decay, be corrected, or be deleted?
7. Which queries over a compartmented metagraph create sensitive derived views even when each source fact is low sensitivity?
8. Which Nostr event kinds and replaceability semantics best fit durable OmegaBuzz definitions and transitions?
9. How much of the dashboard should be implemented in the Buzz desktop client versus a separate web surface?
10. When should cross-hive federation use exported claims, remote queries, or both?
11. How should ontology mappings be adjudicated when two hives deliberately use incompatible conceptual schemes?
12. What minimum source and derivation information makes a Context Bundle portable across models and agent frameworks?
13. Which low-risk action classes recur often enough to justify evidence-driven delegation?
14. How can the system measure useful cognitive diversity without rewarding theatrical disagreement?
15. What metrics predict that an agent roster should be merged, split, reprompted, or retired?

Conclusion

OmegaBuzz can become a practical home for OmegaClaw agents, OmegaHives, and networks of hives because it joins five capabilities that are rarely integrated cleanly:

1. a familiar human-agent collaboration surface;
2. cryptographically attributable event history;
3. explicit organizational control and task ownership;
4. layered semantic memory and uncertain reasoning;
5. provenance-first navigation and context compilation.

The architecture's strength comes from separation rather than fusion. Buzz messages remain messages. Claims remain source-specific and revisable. Tasks have owners. Epics provide loose context without becoming authority. Resident Learners may change through experience without changing organizational reality. Scout finds and packages context without becoming the source of truth. AtomSpace supports semantic cognition without replacing deterministic policy. Hives may share knowledge without sharing all state or converging on one worldview.

Bottom line

Buzz supplies the signed place. OmegaHive supplies the institution. AtomSpace supplies the evolving semantic mind. Scout supplies the inspectable memory map. The Omega Agent Dashboard turns the combination into something a human can actually operate.

The recommended implementation begins with one complete single-hive loop, closes the identified Buzz hardening gaps, and proves source-linked memory promotion and task ownership before attempting Resident Learners or federation. If that vertical slice works, the architecture can expand incrementally into a genuine cognitive operating environment for human-agent organizations.

Canonical Data-Schema Sketches

The following schemas are illustrative. Production contracts should use versioned JSON Schema, Protocol Buffers, or equivalent typed definitions with compatibility tests. Unknown fields should be preserved where safe; unknown schema versions should fail closed for mutations.

A.1 Seat descriptor

```
{
  "$schema": "org.bgi.omegabuzz.seat.v1",
  "seat_id": "seat-macgyver",
  "hive_id": "hive-research-01",
  "display_name": "MacGyver",
  "nostr_pubkey": "...",
  "human_sponsor": "npub-human-owner",
  "role": "practical design partner",
  "charter_ref": "artifact://seat-charter-v4",
  "memory": {
    "private_space": "as://agent/macgyver",
    "shared_spaces": ["as://epic/collective-memory"],
    "promotion_policy": "policy://memory-promotion-v3"
  },
  "participation_mode": "wake-based",
  "default_model_route": "route://frontier-reasoning-qualified",
  "tool_profile": "tools://research-design-v2",
  "permission_tier": 1,
  "budgets": {"daily_tokens": 250000, "daily_usd": 100},
  "pause_authorities": ["npub-human-owner", "seat-assurance"],
  "status": "active"
}
```

A.2 Epic descriptor

```
{
  "$schema": "org.bgi.omegabuzz.epic.v1",
  "epic_id": "epic-collective-memory",
  "title": "Collective Semantic Memory",
  "intent": "Design useful shared AtomSpaces across agents and hives",
  "scope": ["working memory", "durable knowledge", "federation"],
  "exclusions": ["one global unrestricted graph"],
  "stewards": ["npub-human-owner", "seat-bossytron"],
  "participating_hives": ["hive-research-01", "hive-security-01"],
  "open_questions": ["question-1", "question-2"],
  "current_frame_ref": "epicframe://collective-memory/v12",
  "membership_policy": "reviewed-suggestions",
}
```

```

"status": "active",
"next_review_at": "2026-08-07T18:00:00Z"
}

```

A.3 Work order

```

{
  "$schema": "org.bgi.omegabuzz.work-order.v1",
  "work_order_id": "work-441",
  "objective": "Implement stable event anchors in the Scout projector",
  "requester": "seat-bossytron",
  "owner": "seat-codemaxxer",
  "reviewer": "seat-perkins",
  "hive_id": "hive-research-01",
  "epic_ids": ["epic-collective-memory"],
  "inputs": ["adr-17", "schema-source-anchor-v1"],
  "dependencies": ["work-430"],
  "permission_tier": 1,
  "route_requirements": ["code-agent-qualified"],
  "budget": {"tokens": 500000, "usd": 75},
  "expected_outputs": ["pull-request", "contract-tests", "migration-note"],
  "acceptance_tests": ["anchor-stability-suite", "access-filter-suite"],
  "status": "owned",
  "due_at": "2026-08-04T18:00:00Z"
}

```

A.4 Deliberation descriptor

```

{
  "$schema": "org.bgi.omegabuzz.deliberation.v1",
  "deliberation_id": "delib-91",
  "question": "How should foreign claims enter local hive memory?",
  "owning_epic": "epic-collective-memory",
  "room_id": "buzz-room-...",
  "roster": ["npub-human-owner", "seat-xirtus", "seat-macgyver"],
  "chair_policy": "human-chair",
  "synthesizer": "seat-bossytron",
  "visibility": "internal",
  "retention": "durable-transcript",
  "status": "active",
  "opened_event": "event-...",
  "close_by": "2026-08-01T18:00:00Z"
}

```

A.5 Claim

```

{

```

```

"$schema": "org.bgi.omegabuzz.claim.v1",
"claim_id": "claim-882",
"proposition_id": "prop-foreign-import-quarantine",
"predicate": "ForeignClaimRequiresLocalAdmission",
"arguments": ["hive-b", "claim-foreign-19"],
"context": "epic-collective-memory",
"asserted_by": "seat-macgyver",
"source_refs": ["buzz:event-abc", "deliberation:delib-91"],
"epistemic_status": "inferred",
"authority_status": "proposed",
"supporting_evidence": ["artifact-architecture-note"],
"contradicting_evidence": [],
"dependence_groups": ["deliberation-91-source-family"],
"truth_value": {"type": "pln-stv", "strength": 0.86, "confidence": 0.71},
"scope": {"hive": "hive-research-01"},
"freshness": "current",
"visibility": "internal",
"status": "active"
}

```

A.6 Memory promotion request

```

{
  "$schema": "org.bgi.omegabuzz.memory-promotion.v1",
  "request_id": "mpr-72",
  "source_claim_id": "claim-882",
  "source_space": "as://epic/collective-memory",
  "destination_space": "as://hive/research",
  "requested_by": "seat-bossyttron",
  "required_reviews": ["semantic-review", "provenance-review"],
  "policy_version": "memory-promotion-v3",
  "status": "pending"
}

```

A.7 ActionManifest

```

{
  "$schema": "org.bgi.omegabuzz.action-manifest.v1",
  "manifest_id": "manifest-441",
  "requester": "seat-codemaxxer",
  "executor": "worker-deploy-03",
  "source_work_order": "work-441",
  "action_family": "deploy-staging-projector",
  "authorization_basis": "per-action-approval:approval-77",
  "permission_tier": 2,
  "operation": "deploy image sha256:... to staging projector",
  "targets": ["staging-projector-1"],
  "environment": "staging",
  "blast_radius": "one stateless projector instance",
  "forbidden_actions": ["modify production", "read unrelated communities"],
}

```

```
"required_route_state": "local-qualified",  
"rollback_plan": "restore previous image and replay checkpoint",  
"validation_plan": "health, replay, access-control, anchor-stability tests",  
"expires_at": "2026-08-04T20:00:00Z",  
"policy_version": "action-policy-v5",  
"content_hash": "sha256:...",  
"signatures": ["requester-signature", "broker-signature"]  
}
```


Service APIs and Events

B.1 Representative service interfaces

Table B.1: Representative OmegaBuzz APIs

Service	Commands	Queries / events
Hive Registry	CreateHive, UpdateCharter, AddSeat, PauseHive, RetireHive	GetHive, ListHives, HiveChanged, CharterActivated
Seat/Session Manager	CreateSeat, StartSession, DelegateSigner, WakeSeat, EndSession, PauseSeat	GetSeat, SessionStatus, SeatWoken, SessionEnded, SignGrantRevoked
Epic/Task Service	CreateEpic, AssertMembership, CreateWorkOrder, Assign, Block, Complete, Supersede	GetEpicFrame, QueryWork, WorkAged, OwnershipChanged, EpicSynthesized
Deliberation Service	Open, GrantTurn, ProposeSynthesis, Close, Abandon	GetTranscriptRefs, LandingRate, DeliberationOpened, DeliberationClosed
AtomSpace Gateway	AssertPrivateClaim, QueryPattern, LinkEvidence, Snapshot	QueryClaim, QueryConcept, QueryPath, SpaceChanged
Memory Admission	SubmitPromotion, Admit, Quarantine, Reject, Supersede, Retract	PromotionRequired, ClaimAdmitted, ClaimContradicted, MemoryExpired
Archive Projector	RebuildScope, ReindexRange, ApplyRetention, ValidateAnchor	ProjectionStatus, SourceAnchored, ProjectionLag, RedactionApplied
Context Broker	Search, Explain, BuildBundle, ValidateReceipt	SearchResult, BundleCreated, AccessDenied, IndexStale
Policy/Action Broker	EvaluateManifest, RequestApproval, MintCapability, Revoke, Cancel	ApprovalRequired, ManifestDenied, CapabilityIssued, ActionValidated
Federation Gateway	ExportClaims, ImportBundle, MapOntology, RevokeExport	ImportQuarantined, ForeignClaimAvailable, MappingConflict, ExportRevoked
Observability	RecordMetric, OpenIncident, TriggerCircuitBreaker	DashboardSummary, BudgetExceeded, LoopDetected, RecoveryStatus

B.2 API requirements

All mutating APIs should:

- authenticate a human, seat, session, worker, or service identity;

- authorize community, hive, compartment, task, purpose, and time;
- require an idempotency key;
- reject unknown schema versions;
- preserve correlation and causation IDs;
- emit durable events only after validation;
- return typed denials, pending approvals, or partial/degraded states;
- never silently drop unsupported fields.

High-volume internal paths may later use gRPC or another binary protocol, but HTTP/JSON plus durable events is sufficient for the first implementation.

Guidance for Coding Agents

C.1 Development strategy

OmegaBuzz should be built as a collection of small, typed, security-reviewed components rather than one monolithic autonomous application. Coding agents are well suited to parallel implementation of schemas, adapters, projectors, UI components, tests, fixtures, documentation, and migrations. Human architects remain accountable for security boundaries, schema ownership, upstream strategy, release approval, and deployment authority.

Every coding agent receives a narrow work packet, an isolated branch or worktree, explicit acceptance tests, no production credentials, and a named independent reviewer. No agent changes architecture by silently editing code around a failed contract.

C.2 Recommended repository structure

A practical structure is a downstream workspace that keeps upstream Buzz recognizable while placing OmegaBuzz contracts and services in distinct directories.

```
omegabuzz/  
  README.md  
  SECURITY.md  
  GOVERNANCE.md  
  LICENSES/  
  adr/  
  docs/  
    architecture/  
    ux/  
    threat-model/  
    operations/  
    federation/  
  schemas/  
    events/  
    hive/  
    seat-session/  
    epic-task/  
    deliberation/  
    knowledge/  
    context-bundle/  
    action-manifest/  
    federation/  
  crates/  
    omegabuzz-core/  
    omegabuzz-sdk/
```

```
omegabuzz-buzz-adapter/  
hive-registry/  
seat-session-manager/  
epic-task-service/  
deliberation-service/  
wake-governor/  
atomspace-gateway/  
memory-admission/  
archive-projector/  
context-broker/  
key-capability-broker/  
policy-action-broker/  
artifact-registry/  
federation-gateway/  
observability/  
ui/  
  dashboard/  
  agent-setup/  
  hive-view/  
  epic-view/  
  room-panel/  
  scout-integration/  
  claim-explorer/  
  decision-inbox/  
adapters/  
  omegaclaw/  
  openclaw/  
  buzz-acp/  
  scout-substrate/  
  hyperon-atomspace/  
workers/  
  generic-task-worker/  
  code-worker/  
  browser-worker/  
  validation-worker/  
tests/  
  unit/  
  contract/  
  integration/  
  security/  
  workflow/  
  federation/  
  human-factors/  
  resilience/  
fixtures/  
  events/  
  transcripts/  
  hostile-content/  
  atomspaces/  
  federation-bundles/  
deploy/  
  compose/  
  kubernetes/  
  customer-hosted/
```

```
upstream-patches/
buzz/
```

When changes must occur inside the Buzz monorepo, preserve its crate boundaries and add extension hooks rather than importing OmegaBuzz services directly into unrelated Buzz crates.

C.3 Recommended coding-agent roles

Table C.1: Coding-agent roles

Role	Primary work	Mandatory review
Architecture Agent	ADR drafts, boundaries, dependency maps, data flows, failure semantics	Human architect and Security Architecture Agent
Schema Agent	JSON Schema/Protobuf contracts, generated types, compatibility fixtures	API owners and domain-semantic reviewer
Buzz Integration Agent	Event kinds/tags, relay hooks, desktop integration, upstream patch isolation	Upstream-aware maintainer and security reviewer
Identity Agent	Seat/session delegation, key broker interfaces, revocation, owner attestation	Cryptography/security reviewer
Hive/Task Agent	Hive, epic, work-order, deliberation state machines and persistence	Product owner and workflow test agent
AtomSpace Agent	Namespaces, provenance, temporal state, claim graph, query APIs	Hyperon/PLN expert and security reviewer
Memory Admission Agent	Promotion, quarantine, contradiction, dependence groups, retraction	Semantic-policy owner and adversarial test agent
Scout/Projector Agent	Source anchors, substrate generation, retention, Scout integration	Provenance reviewer and access-control reviewer
Context Broker Agent	Hybrid retrieval, authorization, ranking, receipts, bundle compilation	Retrieval evaluator and privacy/security reviewer
Dashboard Agent	Information architecture, fleet/epic/task views, status semantics	Human-factors reviewer and product owner
Resident Learner Agent	Attention scheduler, leases, budgets, loop breakers, charter enforcement	Safety reviewer and empirical-evaluation owner
Policy/Action Agent	Manifest validation, approvals, capability minting, fail-closed logic	Security architect and named human authority
Federation Agent	Export/import, quarantine, ontology mapping, expiry, revocation	Privacy reviewer and semantic interoperability owner
Observability Agent	Metrics, traces, budgets, dashboards, replay and restore checks	SRE/reliability reviewer
Test/Adversarial Agent	Contract, security, hostile-content, isolation, replay, failure injection	Independent human test owner
Release Agent	Dependency pinning, license notices, SBOM, reproducible builds, release notes	Human release authority

C.4 Work-packet contract

Every task assigned to a coding agent should contain:

```

Work Packet ID:
Objective:
Why this matters:
Owning milestone:
Allowed repositories/directories:
Forbidden areas:
Input schemas and ADRs:
External interfaces:
Security assumptions:
Data classification:
Required tests:
Acceptance criteria:
Non-goals:
Expected artifacts:
Independent reviewer:
Budget and timeout:
Escalation conditions:

```

The agent must stop and request clarification when a required schema, security assumption, or ownership boundary is missing. It must not invent a compatible-looking contract and continue silently.

C.5 Branching and review workflow

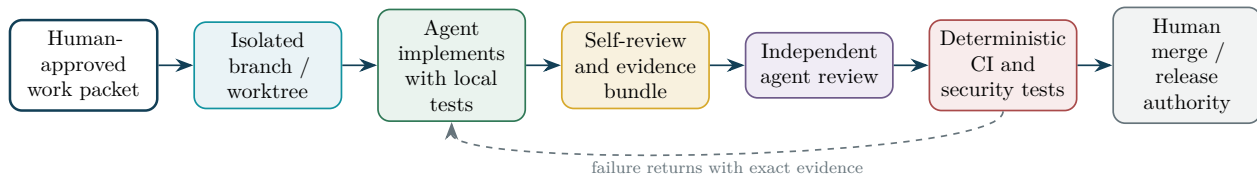


Figure C.1: Coding-agent implementation and review flow.

Requirements:

- no direct pushes to protected branches;
- one work packet per branch unless explicitly grouped;
- schema and migration changes receive independent compatibility review;
- security-boundary changes require human architecture approval before implementation;
- generated code is committed only with its generator and reproducibility instructions;
- every pull request links work packet, ADRs, tests, and evidence;
- failed or incomplete attempts remain visible rather than being summarized away.

C.6 Definition of done

A component is done only when:

1. code compiles and formatting/linting pass;
2. unit tests cover success, denial, and failure paths;
3. contract tests cover every external schema version used;
4. integration tests exercise real dependencies or faithful test containers;
5. security tests cover authorization, tenant/hive isolation, hostile input, and secret handling;
6. idempotency and replay are demonstrated for mutating event flows;
7. metrics, logs, and traces are emitted without sensitive payload leakage;
8. rollback or migration procedure exists;
9. documentation and ADRs are updated;
10. license and dependency notices are correct;
11. independent review is complete;
12. a human accepts the work packet's explicit criteria.

C.7 Continuous integration gates

The first CI pipeline should include:

- Rust formatting, Clippy, unit and integration tests for Buzz-side work;
- TypeScript lint, type checking, component tests, and accessibility checks for UI work;
- JSON Schema validation and backward/forward compatibility fixtures;
- database migration up/down or forward-only safety tests;
- event replay and duplicate-delivery tests;
- community/hive/compartment isolation tests;
- secret scanning, dependency audit, SBOM generation, and license checks;
- hostile-content and prompt-injection fixtures;
- rate-limit, approval-resume, and capability-revocation tests;
- Scout source-anchor and retention tests;
- AtomSpace promotion/quarantine/contradiction tests;
- deterministic build of release artifacts.

C.8 Milestone-specific coding-agent guidance

C.8.1 Milestone M0: baseline and contracts

Coding agents should first inventory current Buzz kind usage, APIs, auth scopes, workflow approval paths, desktop navigation, and audit hooks. They should produce machine-readable inventories and contract tests before changing behavior. The Schema Agent owns the initial object and event contracts; no service begins implementation before those contracts receive review.

C.8.2 Milestone M1: Buzz hardening

Parallel agents may implement:

- production rate limiting with community and identity tiers;
- scoped session authentication and delegated signers;
- workflow approval persistence and resume;
- generic domain-event extension registry;
- broader audit hook API;
- security and E2E tests for each change.

These changes should be isolated into upstreamable pull requests where possible.

C.8.3 Milestone M2: organizational vertical slice

Build Hive Registry, Seat/Session Manager, Epic/Task Service, and Deliberation Service behind typed APIs. Implement the minimal dashboard and room side panel. Use one end-to-end fixture as the acceptance target rather than building every state or UI variation.

C.8.4 Milestone M3: semantic memory and Scout

Implement stable source anchors, Archive Projector, AtomSpace namespaces, claim schema, promotion path, and Context Bundle. Build a fixture that starts from a signed transcript and ends in a promoted claim whose explanation opens exact source events.

C.8.5 Milestone M4: Resident Learner shadow mode

Implement the charter, observation admission, attention scheduler, counterfactual response generation, audit metrics, and human pause. Do not enable unsolicited speech until shadow evaluation passes declared gates.

C.8.6 Milestone M5: multi-hive and federation

Implement same-community namespaces before cross-community transport. Federation work starts with signed offline bundles and deterministic import validation; remote semantic query comes later.

C.9 Agent performance management and recovery

Coding agents themselves should be evaluated by:

- work-packet acceptance rate;
- defects found by independent review and CI;
- regression frequency;
- unsupported architecture changes;
- evidence completeness;
- token and compute cost per accepted change;
- time blocked on missing specification;
- ability to reproduce and repair failures.

When an agent repeatedly fails, the system should preserve its branches and logs, retire or revise the seat/skill/model route, and reassign bounded work. Do not ask the same failing session to review its own redesign.

Initial Engineering Backlog

Table D.1: Initial OmegaBuzz engineering backlog

ID	Work item	Primary role	Depends on	Acceptance summary
OB-001	Freeze core terminology and object model ADR	Architecture Agent	None	Community, hive, seat, session, worker, project, epic, work order, deliberation defined without ambiguity
OB-002	Inventory Buzz event kinds and extension collision risks	Buzz Integration	OB-001	Machine-readable kind map and provisional safe range
OB-003	Define common OmegaBuzz event envelope	Schema Agent	OB-001, OB-002	JSON Schema, examples, compatibility tests
OB-004	Define Hive, Seat, Session schemas	Schema Agent	OB-003	Generated Rust/TypeScript types and fixtures
OB-005	Define Epic, WorkOrder, Deliberation schemas	Schema Agent	OB-003	State transition validation and invalid fixtures
OB-006	Define Claim, Promotion, ContextBundle schemas	Schema Agent	OB-003	Authority/epistemic fields mandatory and tested
OB-007	Define ActionManifest and capability schemas	Policy Agent	OB-003	Content-addressed authorization fields and validation
OB-008	Implement production rate limiter for Buzz	Buzz Integration	None	Community/identity tiers enforced under load
OB-009	Implement scoped NIP-42/session auth	Identity Agent	OB-004	A session receives only declared scopes; negative tests pass
OB-010	Implement workflow approval persistence/resume	Buzz Integration	None	Suspended run resumes exactly once after valid approval
OB-011	Add generic domain-event audit hooks	Buzz Integration	OB-003	OmegaBuzz events appear in verifiable audit chain
OB-012	Implement idempotent projector framework	Architecture Agent	OB-003	Checkpoints, replay, duplicate handling, lag metrics

ID	Work item	Primary role	Depends on	Acceptance summary
OB-013	Implement Hive Registry	Hive/Task Agent	OB-004, OB-012	CRUD through events; charter versions and recovery owners
OB-014	Implement Seat/Session Manager	Identity Agent	OB-004, OB-009, OB-012	Wake/start/end, delegated signer, revocation, audit
OB-015	Implement Epic/Task Service	Hive/Task Agent	OB-005, OB-012	Ownership, dependencies, state machine, aging events
OB-016	Implement Deliberation Service	Hive/Task Agent	OB-005, OB-012	Open/close/abandon, roster, synthesizer, landing rate
OB-017	Add room type and authority banner UI	Dashboard Agent	OB-016	Metadata visible; no hidden authority semantics
OB-018	Build minimal Omega Dashboard	Dashboard Agent	OB-013–016	Attention inbox, active epics, fleet health, current context
OB-019	Implement Artifact Registry	Schema/Artifact Agent	OB-003, OB-012	Hash, classification, provenance, retention, source task
OB-020	Implement stable source-anchor format	Scout Agent	OB-003	Anchors survive regrouping and resegmentation
OB-021	Implement Archive Projector	Scout Agent	OB-012, OB-019, OB-020	Buzz events become Scout substrate with authorization
OB-022	Resolve Scout license or clean-room plan	Human legal/product	None	Written permission/relicense or approved independent implementation
OB-023	Integrate Scout navigation in desktop	Dashboard/Scout	OB-021, OB-022	Open in Scout preserves current scope and source links
OB-024	Implement AtomSpace namespace gateway	AtomSpace Agent	OB-006, OB-013	Scoped agent, epic, hive, quarantine, and policy spaces with ACLs
OB-025	Implement proposition/-claim model	AtomSpace Agent	OB-006, OB-024	Source-specific claims and contradiction links queryable
OB-026	Implement Memory Admission Service	Memory Agent	OB-006, OB-024, OB-025	Promotion, quarantine, supersession, retraction, audit
OB-027	Implement Context Broker v1	Context Agent	OB-021, OB-024	FTS + graph retrieval, result authorization, receipts

ID	Work item	Primary role	Depends on	Acceptance summary
OB-028	Implement Context Cart and bundles	Scout/Context	OB-006, OB-027	Markdown and JSON bundles with signatures and metadata
OB-029	Implement claim explanation UI	Dashboard/AtomSpace	OB-025-028	Why view shows support, contradiction, scope, sources, status
OB-030	Implement Key/Capability Broker	Identity/Policy	OB-007, OB-009	Long-term keys isolated; short-lived scoped grants revocable
OB-031	Implement Policy/Action Broker shadow mode	Policy Agent	OB-007, OB-030	Typed decisions, no execution, complete audit
OB-032	Implement bounded worker execution	Worker Agent	OB-031	One sandbox action with capability, rollback, validation
OB-033	Implement Agent Setup Wizard	Dashboard Agent	OB-013, OB-014, OB-024, OB-030	Seat can be configured and test-activated end to end
OB-034	Implement Resident Learner charter and shadow mode	Resident Agent	OB-014, OB-024	Observe and generate counterfactual speech without posting
OB-035	Implement speech leases and loop breakers	Resident Agent	OB-034	Duplicate/ack loops suppressed; substantive disagreement preserved
OB-036	Implement multi-hive namespaces	Architecture/AtomSpace	OB-013, OB-024	Two hives isolated within one community
OB-037	Implement joint epics and cross-hive rooms	Hive/Task Agent	OB-015, OB-016, OB-036	Shared context without merged ownership
OB-038	Define federation bundle schema	Federation/Schema	OB-006, OB-036	Source, ontology, evidence, expiry, revocation fields
OB-039	Implement offline export/import quarantine	Federation Agent	OB-026, OB-038	Foreign claim preserved and cannot auto-admit
OB-040	Implement ontology mapping review	Federation/AtomSpace	OB-039	Lossy/ambiguous mappings remain explicit
OB-041	Build end-to-end vertical-slice test	Test Agent	OB-013-032	Deliberation to task to claim to Scout to dashboard passes
OB-042	Build isolation and leakage test suite	Security Test Agent	OB-021, OB-024, OB-027, OB-036	No cross-hive/compartment search, graph, cache, or bundle leakage
OB-043	Build event replay and restore suite	Reliability Agent	OB-012, OB-024, OB-026	Fresh install rebuilds state and reports unreplicable items

ID	Work item	Primary role	Depends on	Acceptance summary
OB-044	Build human-factors pilot protocol	Product/HF Agent	OB-018, OB-023, OB-029, OB-033	Users locate decisions, understand status, and operate fleet
OB-045	Produce upstream contribution plan	Architecture/ Human	OB-008-011	Patch ownership, rebasing policy, upstream PR sequence

Testing Matrix

E.1 Test levels

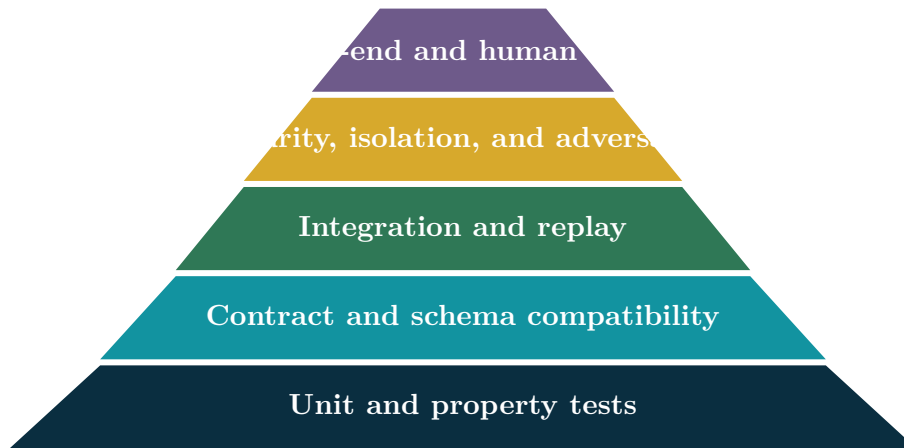


Figure E.1: OmegaBuzz test pyramid. Security and human factors are explicit levels, not afterthoughts.

Table E.1: Testing matrix

Level	Scope	Critical assertions
Unit	Schema validators, state predicates, permission calculations, source anchors, metric functions	Invalid transitions rejected; unknown not coerced into false; confidence does not imply authority; anchor hashes deterministic
Contract	Buzz events, Nostr tags, APIs, AtomSpace adapters, Scout substrate, model routes	Unknown versions fail closed for writes; fields survive translation; rate limits and partial responses are typed
Integration	Relay, projectors, task service, AtomSpace, Scout, policy, workers	Duplicate delivery idempotent; quarantined memory cannot influence durable views; complete traces maintained
Security	Community/hive isolation, key broker, prompt injection, embeddings, derived views, sandboxes	No cross-scope retrieval; no long-term key exposure; content cannot grant authority; scope escape denied
Workflow	Deliberation, synthesis, task, memory promotion, action, rollback, federation	End-to-end state matches expected history; failure/degradation cannot become completion or admission

Level	Scope	Critical assertions
Resident learner	Attention, speech, loop breakers, privacy, memory provenance	Shadow output measured; leases prevent parallel chatter; pause immediate; source-linked social memory
Federation	Export/import, ontology mapping, expiry, revocation	Foreign authority preserved; no auto-admission; revocation and stale state visible
Resilience	Projector outage, route degradation, DB restore, snapshot replay, audit loss	Degrades honestly, fails closed for consequential writes, recovers without agent-only access
Human factors	Dashboard, setup wizard, decision packets, Scout navigation	Users understand ownership, authority, uncertainty, and can locate decisive sources quickly

E.2 Critical adversarial fixtures

The test corpus should include:

- a chat message claiming to grant itself production authority;
- a malicious document instructing an agent to ignore its work order;
- two agents repeating the same source as independent evidence;
- a stale favorable claim arriving just before memory promotion;
- a context-bundle request that crosses compartments through semantic similarity;
- a foreign hive export with ambiguous ontology mapping;
- a replaced or revoked seat key replaying an old capability;
- an approval changed after the manifest hash was signed;
- a projector that crashes after updating one store but before checkpointing;
- a Resident Learner acknowledgment loop and a substantive disagreement that superficially resembles repetition;
- a route that remains available while structured-output quality degrades;
- a deleted source whose embedding remains in a cache;
- an epic membership suggestion that attempts to reassign a task owner;
- a valid proof or test artifact attached to the wrong deployed object;
- an event replay containing duplicates and out-of-order delivery.

Glossary

Term	Meaning
AtomSpace	Hypergraph/metagraph knowledge representation used for structured semantic memory and reasoning.
Community	Buzz relay-facing administrative and tenant-visible trust domain selected by host/URL.
Context Broker	Authorization-aware service combining lexical, vector, graph, task, and temporal retrieval.
Context Bundle	Signed, source-linked package of selected context for a human or agent session.
Deliberation	Bounded conversation opened around a question and closed into committed objects or explicit abandonment.
Epic	Loosely bounded evolving intention-context linking work, rooms, claims, and artifacts across projects or hives.
EpicFrame	Compact reviewable synthesis of an epic's intent, questions, decisions, work, evidence, and changes.
Federation Gateway	Service for signed knowledge export/import, quarantine, ontology mapping, local admission, expiry, and revocation.
Hive	Persistent governed human-agent organizational and cognitive unit.
Memory Admission	Reviewed path by which proposed or imported claims enter durable shared knowledge.
OmegaClaw	AtomSpace-backed cognitive agent in the broader Hyperon/OmegaHive architecture.
OpenClaw	Skill-executing or tool-oriented agent used as a bounded worker or specialist.
Resident Learner	Agent allowed ongoing ambient observation and developmental memory under a separate governor.
Scout	Provenance-first archive reader and context selection/bundling interface.
Seat	Durable agent role, public identity, charter, memory lineage, and contribution history.
Session	One bounded activation of a seat with a model route, context, budget, and delegated signer.
Worker	Narrow task-scoped execution process with limited capabilities and task-local state.
WorkOrder	Explicitly owned unit of non-trivial work with state, dependencies, budget, outputs, and acceptance tests.

References

Bibliography

- [1] Block, Inc. *Introducing Buzz: where humans and agents work together*. 2026. Accessed 30 July 2026. <https://block.xyz/inside/introducing-buzz-where-humans-and-agents-work-together>
- [2] Block, Inc. *Buzz: A workspace where humans and agents build together, on a relay you own*. GitHub repository. Accessed 30 July 2026. <https://github.com/block/buzz>
- [3] Block, Inc. *Buzz Architecture*. Public repository document, current main branch as reviewed 30 July 2026. <https://github.com/block/buzz/blob/main/ARCHITECTURE.md>
- [4] Block, Inc. *Buzz Projects - A Nostr-Native Forge*. Public vision document, reviewed 30 July 2026. https://github.com/block/buzz/blob/main/VISION_PROJECTS.md
- [5] Block, Inc. *Vision: buzz-agent + buzz-dev-mcp*. Public vision document, reviewed 30 July 2026. https://github.com/block/buzz/blob/main/VISION_AGENT.md
- [6] Block, Inc. *Using Third-Party Nostr Clients with Buzz*. Public repository document, reviewed 30 July 2026. <https://github.com/block/buzz/blob/main/NOSTR.md>
- [7] Dianne Krouse. *Scout: A persistent memory and context layer for years of conversations, documents, and notes*. GitHub repository, reviewed 30 July 2026. <https://github.com/diannekrouse/Scout>
- [8] Dianne Krouse. *Scout Feature Inventory*. Public repository document, reviewed 30 July 2026. <https://github.com/diannekrouse/Scout/blob/main/FEATURES.md>
- [9] Dianne Krouse and SingularityNET Foundation. *Scout LICENSE file*. Public repository document, reviewed 30 July 2026. <https://github.com/diannekrouse/Scout/blob/main/LICENSE>
- [10] Ben Goertzel. *OmegaHive1: An Experimental Cooperative Hive of OmegaClaw and OpenClaw Agents*. Internal design brief, 2026.
- [11] Cassio Pennachin. *OmegaHive Deliberation Rooms: Vision, a Slack MVP, and the relation to the Conversation Governor*. Internal design note, 29 July 2026.
- [12] Benjamin Goertzel, ProtoCosmoBot, ProtomegaBot, and ZeroBot. *Resident Learners in OmegaHive: A minimal conversation governor on top of wake-based deliberation rooms*. Internal design synthesis, 30 July 2026.
- [13] Ben Goertzel. *Omega Purple: Agentic Security Office - Product Overview and Design Specification 1.2*. BGI Labs internal product and architecture specification, 29 July 2026.
- [14] BGI Labs. *BGI Labs Seed Round Deck, version 2*. Internal presentation, 2026.